INSTITUTO TECNOLÓGICO DE AERONÁUTICA

João Matheus Del Vecchio França Barbosa

HAWKES PROCESS ESTIMATION ON HIGH-FREQUENCY CRYPTOCURRENCY DATA VIA NEURAL NETWORKS

Trabalho de Graduação
2026

Curso de Engenharia de Computação

João Matheus Del Vecchio França Barbosa

HAWKES PROCESS ESTIMATION ON HIGH-FREQUENCY CRYPTOCURRENCY DATA VIA NEURAL NETWORKS

Orientador
Prof. Dr. Mauri Aparecido de Oliveira (ITA)

ENGENHARIA DE COMPUTAÇÃO

SÃO JOSÉ DOS CAMPOS
INSTITUTO TECNOLÓGICO DE AERONÁUTICA

2026

HAWKES PROCESS ESTIMATION ON HIGH-FREQUENCY CRYPTOCURRENCY DATA VIA NEURAL NETWORKS

Essa publicação foi aceita como Relatório Final de Trabalho de Graduação

João Matheus Del Vecchio França Barbosa
Autor

Prof. Dr. Mauri Aparecido de Oliveira (ITA)
Orientador

São José dos Campos, XX de XXXXX de 2026

Resumo

Processos de Hawkes descrevem sequências de eventos em que cada ocorrência eleva temporariamente a probabilidade de novas ocorrências; em microestrutura, representam o agrupamento de alterações de preço. Este trabalho reproduz e estende a estimação amortizada, por redes neurais, dos parâmetros de um Hawkes bidimensional simétrico proposta por Lee (2023), com dados de alta frequência dos futuros perpétuos BTCUSDT e ETHUSDT da Binance. Uma pipeline reproduzível reconstrói o mid-price em grade de 100 ms, simula trajetórias por *thinning*, estima por máxima verossimilhança (MLE) e treina a LSTM do artigo e um encoder Transformer. No protocolo sintético congelado (100.000 trajetórias de 1.000 eventos, cinco seeds por rede), a LSTM obteve MSE macro de 0,0196 e o Transformer, 0,0237 ($p = 0{,}19$). O MSE do MLE (0,506) é dominado por 1% das trajetórias, nas quais β é pouco identificado; na trajetória típica, o MLE é tão preciso quanto as redes e, com gradiente analítico, custa cerca de 10 ms, a mesma ordem das redes em lote unitário. Nas janelas reais, o Hawkes supera o Poisson em 99,8% das janelas, mas deixa dependência nos resíduos, e os parâmetros ficam fora do suporte sintético, com excitação cruzada nula. Redes treinadas com rótulos MLE de janelas reais, com ajuste fino a partir dos pesos sintéticos, atingem MSE de teste de 0,044 (LSTM, escolhida na validação) e 0,040 (Transformer), contra 0,218 da média dos rótulos; no teste não se distinguem, e numa janela de outubro de 2023 o Transformer degrada menos. Num adendo pré-registrado, avaliado em semanas novas, a LSTM treinada por mais épocas reduz o erro em 20% (0,026 para 0,021, fevereiro de 2024) e, pré-treinada num prior construído a partir dos rótulos reais, em mais 63% (0,109 para 0,040, março de 2024); a grade de 100 ms faz o MLE estimar cerca de 58% do β em tempo contínuo, viés que uma rede desfaz em dados simulados, mas não com segurança nos reais. A volatilidade de Hawkes do MLE ordena as janelas como a volatilidade realizada (Spearman 0,78); a da rede preserva o nível, mas não a ordenação (0,28). Conclui-se que a atenção não superou a LSTM nos dados sintéticos, que a vantagem da estimação amortizada é a robustez e o custo em lote, e que, em cripto, o treino sintético serve de ponto de partida para o ajuste fino e rende mais quando o prior vem dos próprios dados.

Palavras-chave: processos de Hawkes; alta frequência; criptomoedas; LSTM; Transformer; estimação amortizada.

Abstract

Hawkes processes describe event sequences in which each occurrence temporarily raises the probability of further occurrences; in market microstructure they represent the clustering of price changes. This work reproduces and extends the amortized neural estimation of the parameters of a symmetric bivariate Hawkes process proposed by Lee (2023), using high-frequency data from Binance BTCUSDT and ETHUSDT perpetual futures. A reproducible pipeline reconstructs the mid-price on a 100 ms grid, simulates paths by thinning, estimates parameters by maximum likelihood (MLE), and trains the paper's LSTM and a Transformer encoder. Under the frozen synthetic protocol (100,000 paths of 1,000 events, five seeds per network), the LSTM reached a macro MSE of 0.0196 and the Transformer 0.0237 ($p = 0.19$). The MLE macro MSE (0.506) is dominated by 1% of the paths, in which β is weakly identified; on a typical path the MLE is as accurate as the networks and, with an analytic gradient, costs about 10 ms, the same order as the networks at batch size one. On real windows, the Hawkes model beats a Poisson process in 99.8% of the windows but leaves residual dependence, and the parameters lie outside the synthetic support, with null cross-excitation. Networks trained on MLE labels of real windows, fine-tuned from the synthetic weights, reach a test macro MSE of 0.044 (LSTM, selected on validation) and 0.040 (Transformer), against 0.218 for the label mean; they cannot be told apart on the test block, and in an October 2023 window the Transformer degrades less. In a pre-registered addendum evaluated on new weeks, the LSTM trained for more epochs lowers the error by 20% (0.026 to 0.021, February 2024) and, pretrained on a prior built from the real labels, by a further 63% (0.109 to 0.040, March 2024); the 100 ms grid makes the MLE estimate about 58% of the continuous-time β, a bias that a network undoes on simulated data but not safely on real data. The Hawkes volatility implied by the MLE ranks the windows like the realized volatility (Spearman 0.78); the network's keeps the level but not the ranking (0.28). We conclude that attention did not outperform the LSTM on synthetic data, that the benefit of amortized estimation is robustness and batched cost, and that on crypto data the synthetic training is most useful as the starting point for fine-tuning, and more so when its prior is built from the data.

Keywords: Hawkes processes; high frequency; cryptocurrencies; LSTM; Transformer; amortized estimation.

List of Figures

2.1 Intensity of a univariate Hawkes process with $\mu = 0.5$, $\alpha = 0.8$ and $\beta = 2$, for three events (ticks on the axis). Each event adds a jump that decays with half-life $\ln 2/2 \approx 0.35$ s. The dots are the intensities at the events, the terms that the likelihood rewards; the shaded area is the compensator, the term it penalizes. . . 17

2.2 Cluster representation of a Hawkes process. Only the projection of the events on the time axis is observed; which event triggered which is hidden. With $g = 0.5$, each immigrant brings on average a cluster of $1/(1 - 0.5) = 2$ events. 17

2.3 The symmetric bivariate Hawkes process as a diagram of event types. Loops are self-excitation, arrows between types are cross-excitation; the same four parameters $(\mu, \alpha_{\text{self}}, \alpha_{\text{cross}}, \beta)$ apply to both types. $\alpha_{\text{self}} > \alpha_{\text{cross}}$ describes persistence of direction, as found in the crypto data of this work; $\alpha_{\text{cross}} > \alpha_{\text{self}}$ describes reversion, as at the equity fixed point of Lee (2023). 18

6.1 Test MSE by parameter under the 1,000-event protocol. The network error bars represent one standard deviation across seeds. 38

6.2 Macro MSE by regime of the branching ratio. The network error bars represent one standard deviation across seeds. 38

6.3 Cost per path recorded during the final run, on a logarithmic scale. For the MLE, wall-clock time with four processes competing with training on the GPU; for the networks, median latency at batch size one on MPS. A later controlled benchmark measured about 10 ms per path for the MLE on a single core and should be preferred to this figure. 39

6.4 True versus estimated β (3,000 test paths, logarithmic scale). The networks shrink the estimates toward the center of the prior; the MLE is unbiased over most of the domain but produces extreme values when g is low. 40

6.5 MLE estimates on the real calibration windows (points) and support of the prior used to generate the training data (rectangles). 41

6.6 Left: Q–Q plot of the pooled time-rescaled residuals against Exp(1). Right: rejection rates on real data and in the two simulated controls; the dotted line is the nominal 5%. 44

6.7 Left: median realized volatility per $\sqrt{s}$ as a function of the sampling interval (signature plot) and the median Hawkes level. Right: Hawkes against realized volatility (Δ = 10 s) per window. 46

List of Tables

3.1 Positioning of this work relative to the closely related literature. 29

4.1 Dataset coverage. 30
4.2 Characteristics of the mid-price events, January 15 to 22, 2024. 31

5.1 Training configuration and cost (five seeds per network, Apple Silicon Mac with MPS). 33

6.1 Sampling distribution at the fixed point of Lee (2023) ($\mu = 0.3$, $\alpha_{\text{self}} = 0.4$, $\alpha_{\text{cross}} = 0.7$, $\beta = 1.5$): mean and standard deviation over 2,000 paths of 1,000 events. Lee's rows use 100,000 paths of 2,000 steps. 36
6.2 Aggregate metrics on the frozen synthetic test set (15,000 paths of 1,000 events). 37
6.3 Test MSE by parameter (networks: mean over five seeds). 37
6.4 Macro MSE by regime of the branching ratio g. 37
6.5 Per-parameter R^2 relative to the prior mean, macro median absolute error and MSE of the branching ratio on the synthetic test set. 39
6.6 Paired comparison by path (mean standardized squared error over the four parameters). Columns: median loss of A and of B, fraction of paths in which A has the smaller error, and *bootstrap* 95% CI of the mean difference A−B (negative favors A). 40
6.7 Cost per path of 1,000 events in a controlled benchmark. 42
6.8 Cost of one MLE fit by implementation, at the fixed point of Lee (2023): median of 10 paths, one start at the true parameters, one core. The pure-Python double sum was timed for one evaluation and multiplied by the number of evaluations of the vectorized Nelder–Mead fit. 42
6.9 Residual diagnostics per window and dimension. Under a correct model, about 5% of the tests reject at the 5% level. 43

6.10 Agreement with the MLE label on real windows: macro MSE of the seed ensemble and mean ± standard deviation across seeds on the validation block, and R^2 per parameter relative to the training-label mean (share of the label variance reproduced). *Selected variant. 45
6.11 Hawkes volatility against realized volatility: median ratio σ_H/σ_{RV} and Spearman correlation across windows, by sampling interval Δ. 45
6.12 Agreement with the MLE label in the October 2023 robustness window: macro MSE of the seed ensemble, overall and by tercile of realized volatility. *Variant selected on the January validation block. 46
6.13 Mean loss against the MLE on simulated paths, ensemble of five seeds. The first row is the true β of the simulation; * marks values added after the registered grid. 47
6.14 Median MLE β on the 100 ms grid for each continuous-time (μ, β), $g = 0.5$, 100 paths per cell. 48
6.15 Validation macro MSE of the five-seed ensembles. The selection among the new arms was recorded before the new week was read. 49
6.16 Macro MSE and R^2 per parameter on the new week, and macro MSE on October 2023 (exploratory), ensembles of five seeds. R^2 of α_c is left out of the table because its label variance is close to zero, which makes it unstable; it is discussed in the text. 50
6.17 Secondary paired comparisons on the new week (difference A minus B; negative favours A). The interval is for the mean difference and the p-value is Wilcoxon's, which looks at the median, so the two can disagree when a few windows dominate the mean. 50
6.18 Macro MSE of the five-seed ensembles. Validation was recorded before the March week was read; February and October had been read before and are exploratory. 51
6.19 Error against the continuous-time parameters on 2,000 new simulated paths. Relative error of β: signed and absolute medians. 53
6.20 Spearman correlation between the Hawkes volatility and the realized volatility across windows, and median ratio of the two. 54

C.1 Integration tests prior to the frozen protocol. 63

Contents

1 Introduction **12**

1.1 Context . . . 12

1.2 Problem . . . 12

1.3 Hypotheses . . . 13

1.4 Objectives . . . 13

1.5 Contributions . . . 13

2 Theoretical Background **15**

2.1 Point processes . . . 15

2.2 Poisson as the null model . . . 16

2.3 Univariate Hawkes process . . . 16

2.4 Symmetric bivariate Hawkes process . . . 18

2.5 Stability . . . 19

2.6 Marks and price . . . 19

2.7 Volatility . . . 19

2.8 Simulation . . . 21

2.9 Maximum likelihood . . . 21

2.10 Model comparison . . . 22

2.11 Goodness of fit by time rescaling . . . 22

2.12 Observation on a time grid . . . 22

2.13 Identifiability in finite samples . . . 23

2.14 Amortized estimation as a Bayes estimator . . . 23

2.15 Comparing estimators . . . 24

2.16 LSTM . . . 25

2.17 Attention and the Transformer . . . 25

2.18 Training . . . 26

3 Related Work **27**

3.1 Hawkes processes in finance . . . 27

3.2 Estimation of Hawkes processes . . . 27
3.3 Amortized and simulation-based inference . . . 27
3.4 Neural point processes . . . 28
3.5 Hawkes processes in crypto-assets . . . 28
3.6 Positioning . . . 28

4 Data **30**
4.1 Source and market . . . 30
4.2 Schema . . . 30
4.3 Sampling . . . 30
4.4 Datasets produced . . . 30
4.5 Event characteristics . . . 30

5 Methodology **32**
5.1 Principles . . . 32
5.2 Pipeline . . . 32
5.3 Simulation by thinning . . . 32
5.4 Constrained MLE . . . 33
5.5 Neural protocol . . . 33
5.6 The paper's fixed point (E2) . . . 34
5.7 Empirical protocol . . . 34
5.8 Metrics and tests . . . 35

6 Results **36**
6.1 Data gate . . . 36
6.2 Reproduction at the fixed point . . . 36
6.3 Final comparison: LSTM, Transformer and MLE . . . 36
6.4 Complementary analysis of the synthetic results . . . 39
6.5 Empirical calibration and support of the synthetic prior . . . 41
6.6 Computational cost . . . 41
6.7 Empirical calibration . . . 43
6.8 Goodness of fit on real data . . . 43
6.9 Neural estimators on real windows . . . 44
6.10 Hawkes volatility . . . 44

6.11 Robustness window 46
6.12 Addendum: analyses registered after the results were frozen 47
6.12.1 Does the LSTM degrade faster when β moves? 47
6.12.2 How much of β does the 100 ms grid hide? 48
6.12.3 A better fine-tuning recipe 49
6.12.4 A new week as a clean test 49
6.12.5 A synthetic prior built from the real labels (reduced scope B) 51
6.12.6 Can a network undo the bias of the grid? 52
6.12.7 Hawkes volatility with the addendum networks 53

7 Discussion **55**
7.1 Assessment of the hypotheses 55
7.2 Why the networks win on aggregate MSE 56
7.3 Comparison with Lee (2023) 56
7.4 What the real data show 56
7.5 Threats to validity 58

8 Future Work **59**

9 Conclusion **60**

A Ambiguities in Lee (2023) **61**

B Computational Reproduction **62**

C Pipeline Validation History **63**

1 Introduction

1.1 Context

Financial prices do not change at regular intervals. At short time scales, one observes a sequence of events: an upward change, a downward change, a trade, the addition or removal of an order. Activity alternates between calm periods and bursts. This temporal dependence is called *clustering* and is linked to volatility (Bouchaud; Bonart; Donier; Gould, 2018).

A homogeneous Poisson process assumes a constant rate and independent increments. This assumption is useful as a starting point, but it cannot represent the fact that a recent change tends to be followed by further changes. The Hawkes process (Hawkes, 1971b; Hawkes, 1971a) removes this limitation by making the intensity depend on its own history, and it has become a reference model in high-frequency finance (Bacry; Mastromatteo; Muzy, 2015; Hawkes, 2018).

The practical challenge is estimating the parameters. MLE has a clear statistical foundation, but it requires iterative optimization for each new window. Lee (2023) proposes paying a training cost once and using an LSTM to map the observed sequence directly to the parameters. This strategy is *amortized inference*: many future inferences reuse the learned knowledge (Cranmer; Brehmer; Louppe, 2020).

1.2 Problem

The reference paper uses United States equities between 2018 and 2019. Its NBBO data were not published, and important details of the protocol are missing. Moreover, only one compact recurrent architecture was studied. Crypto-assets offer a natural test case: continuous trading, 24 hours a day, and public best bid/offer data (Makarov; Schoar, 2020). The present work asks:

> Can an attention-based architecture estimate the parameters of a 2D Hawkes process on cryptocurrency price movements with accuracy and stability equal to or better than Lee's LSTM, while preserving faster inference than MLE?

1.3 Hypotheses

The hypotheses were registered before the final run and are kept as originally written:

H1. Binance's public USD-M data make it possible to build sequences structurally equivalent to Lee's inputs.

H2. The LSTM achieves error of the same order as MLE with substantially lower latency.

H3. Attention improves error or robustness on sequences with long-range dependencies.

H3 may be rejected. A negative result, if obtained under a frozen protocol, is scientifically informative.

1.4 Objectives

The general objective is to reproduce and extend neural Hawkes estimation on crypto. Specific objectives:

1. build and audit the top-of-book dataset;
2. implement the 2D Hawkes process, simulation and MLE;
3. reproduce the paper's LSTM;
4. adapt the pipeline to BTCUSDT and ETHUSDT;
5. design a comparable Transformer;
6. evaluate accuracy, validity, cost and robustness;
7. apply the best estimator to Hawkes volatility.

Objectives 4 and 7 are addressed by the empirical stage (Sections 6.8 to 6.11).

1.5 Contributions

The first contribution concerns reproduction: open-source code, a record of the paper's gaps, and a quantitative reproduction of the sampling distribution at the published fixed point. The second is methodological: an explicit separation between synthetic reproduction, empirical adaptation and extension, with paired statistical tests and robust metrics that show why the aggregate MSE favors the networks. The third is architectural: a fair comparison of LSTM and Transformer, with a negative result for attention. The fourth is empirical: a characterization of the mid-price events of BTCUSDT and ETHUSDT, evidence that their estimated parameters lie outside the typical region for equities, a goodness-of-fit diagnosis that separates the effect of the 100 ms

grid from misspecification, a pre-registered evaluation of networks trained on MLE labels of real windows, and a comparison of Hawkes volatility with realized volatility.

2 Theoretical Background

This chapter presents the model and the tools used in the rest of the work: the Hawkes process, from the univariate case to the symmetric bivariate form used here, its stability and the volatility it implies; simulation, maximum likelihood, model comparison and the goodness-of-fit test built on the compensator; the effect of observing the events on a time grid; the reading of a network trained on simulated data as a Bayes estimator; the statistical tools used to compare estimators; and the two architectures compared, with their training.

2.1 Point processes

If $t_1 < t_2 < \cdots$ are event times, the counting process is

$$N(t) = \sum_k \mathbf{1}_{\{t_k \le t\}}.$$

The conditional intensity $\lambda(t \mid \mathcal{H}_t)$ describes the expected instantaneous rate given the history $\mathcal{H}_t$ (Daley; Vere-Jones, 2003). Informally,

$$\Pr[N(t + \mathrm{d}t) - N(t) = 1 \mid \mathcal{H}_t] \approx \lambda(t)\,\mathrm{d}t.$$

It is not a probability in arbitrary units: intensity has the dimension of events per unit of time. All processes in this work are simple: two events never occur at the same instant, so the probability of more than one event in $\mathrm{d}t$ is of smaller order. The intensity then determines the law of the process. Its integral,

$$\Lambda(t) = \int_0^t \lambda(u)\,\mathrm{d}u,$$

is the compensator: the number of events expected up to t given the past, so that $N(t) - \Lambda(t)$ is a martingale. The compensator appears twice below, in the likelihood (Section 2.9) and in the goodness-of-fit test (Section 2.11).

2.2 Poisson as the null model

In the homogeneous Poisson process, $\lambda(t) = \lambda_0$. Inter-event times are independent exponentials. The model produces no bursts beyond those due to chance. It is therefore a useful control: if Hawkes does not outperform Poisson in diagnostics, self-excitation is not justified. In the inhomogeneous Poisson process the intensity is a deterministic function $\lambda_0(t)$, so it can follow, for example, an intraday pattern of activity, but it still ignores the past. The Hawkes process keeps the dependence on the past and lets each event change the rate of the events that follow.

2.3 Univariate Hawkes process

With an exponential kernel,

$$\lambda(t) = \mu + \sum_{t_k < t} \alpha e^{-\beta(t - t_k)}. \tag{2.1}$$

μ is the exogenous activity; α is the immediate jump; β controls the speed of forgetting. The integral of the kernel is α/β, the branching ratio (Laub; Lee; Taimre, 2021). Each event raises the intensity by α, and the increase decays with half-life $\ln 2/\beta$ (Figure 2.1).

In general, $\lambda(t) = \mu + \int_{(0,t)} \phi(t - s)\, \mathrm{d}N(s)$ for a non-negative kernel ϕ (Hawkes, 1971b). The exponential kernel $\phi(s) = \alpha e^{-\beta s}$ has a single time scale. A sum of exponentials, $\sum_m \alpha_m e^{-\beta_m s}$, mixes several scales and keeps the recursion below for each term. A power law, $\phi(s) \propto (c + s)^{-(1+\gamma)}$, has long memory, and fits of this kind to financial data tend to place the process close to criticality (Hardiman; Bercot; Bouchaud, 2013; Bacry; Mastromatteo; Muzy, 2015). The choice matters for the diagnostics: a single exponential fitted to data with several time scales leaves dependence in the residuals (Section 2.11).

The intensity as a state. Write $E(t) = \lambda(t) - \mu$ for the excitation. Between events, E decays as $\mathrm{d}E/\mathrm{d}t = -\beta E$; at an event it jumps by α. Hence, with $\Delta_k = t_k - t_{k-1}$,

$$E(t_k^-) = e^{-\beta \Delta_k} \big(E(t_{k-1}^-) + \alpha \big), \tag{2.2}$$

and the current intensity summarizes the whole past: with an exponential kernel the process $\lambda(t)$ is Markov. This property makes simulation and the likelihood linear in the number of events. It does not hold for other kernels, such as the power law, where the whole history must be kept.

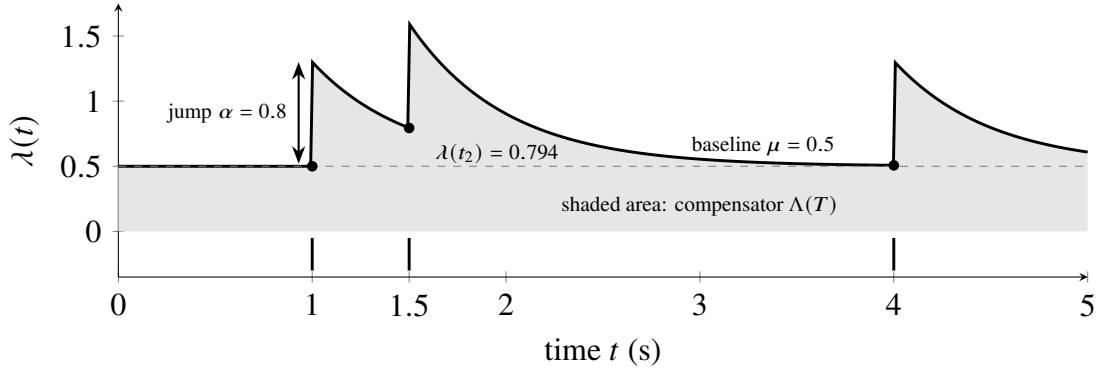


Figure 2.1 – Intensity of a univariate Hawkes process with $\mu = 0.5$, $\alpha = 0.8$ and $\beta = 2$, for three events (ticks on the axis). Each event adds a jump that decays with half-life $\ln 2/2 \approx 0.35$ s. The dots are the intensities at the events, the terms that the likelihood rewards; the shaded area is the compensator, the term it penalizes.

Cluster representation. The same process can be generated as a population (Hawkes, 2018; Laub; Lee; Taimre, 2021). Immigrants arrive as a Poisson process of rate μ; each event, immigrant or not, produces children as an inhomogeneous Poisson process of rate $\alpha e^{-\beta s}$, where s is the time since the parent. The number of direct children of an event is Poisson with mean

$$g = \int_0^\infty \alpha e^{-\beta s}\,\mathrm{d}s = \frac{\alpha}{\beta},$$

which is why g is called the branching ratio. Children, grandchildren and further generations form a geometric series, so the mean size of the cluster started by one immigrant is

$$1 + g + g^2 + \cdots = \frac{1}{1-g} \qquad (g < 1),$$

and a fraction g of all events is endogenous, triggered by earlier events rather than by the baseline. Taking expectations in Equation (2.1), the stationary mean rate $\bar{\lambda} = \mathbb{E}[\lambda(t)]$ satisfies $\bar{\lambda} = \mu + g\bar{\lambda}$, that is, $\bar{\lambda} = \mu/(1-g)$. Only the times are observed; the genealogy (Figure 2.2) is latent.

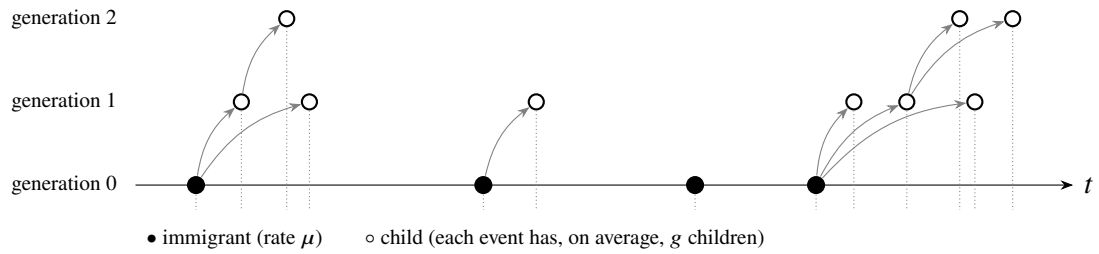


Figure 2.2 – Cluster representation of a Hawkes process. Only the projection of the events on the time axis is observed; which event triggered which is hidden. With $g = 0.5$, each immigrant brings on average a cluster of $1/(1-0.5) = 2$ events.

2.4 Symmetric bivariate Hawkes process

A D-dimensional Hawkes process has one intensity per type,

$$\lambda_i(t) = \mu_i + \sum_{j=1}^{D} \int_{(0,t)} \phi_{ij}(t-s)\, \mathrm{d}N_j(s), \qquad i = 1, \ldots, D,$$

where ϕ_{ij} is the effect of a type-j event on the intensity of type i (Hawkes, 1971a; Bacry; Mastromatteo; Muzy, 2015). With exponential kernels and two types there are, in general, two baselines, four jump sizes and up to four decay rates. Here, up-moves and down-moves are processes N_1, N_2 with a common decay and symmetric excitation:

$$\lambda_1(t) = \mu + \sum_{t_k^1 < t} \alpha_{\text{self}} e^{-\beta(t-t_k^1)} + \sum_{t_k^2 < t} \alpha_{\text{cross}} e^{-\beta(t-t_k^2)}, \tag{2.3}$$

$$\lambda_2(t) = \mu + \sum_{t_k^2 < t} \alpha_{\text{self}} e^{-\beta(t-t_k^2)} + \sum_{t_k^1 < t} \alpha_{\text{cross}} e^{-\beta(t-t_k^1)}. \tag{2.4}$$

The model reduces a general matrix to four parameters. This parsimony makes simulation and comparison with Lee (2023) easier. Figure 2.3 draws it as a diagram of the two event types. An up-move raises the intensity of another up-move by α_{self} and that of a down-move by α_{cross}, and symmetrically for a down-move; both increases decay at the same rate β. In microstructure, dominant cross-excitation indicates reversion (an up-move followed by a down-move), and dominant self-excitation indicates persistence of direction (Bacry; Delattre; Hoffmann; Muzy, 2013).

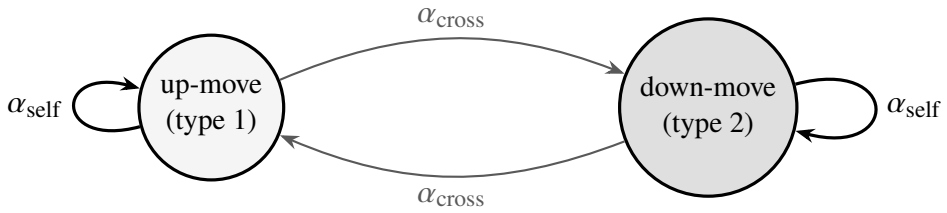


Figure 2.3 – The symmetric bivariate Hawkes process as a diagram of event types. Loops are self-excitation, arrows between types are cross-excitation; the same four parameters $(\mu, \alpha_{\text{self}}, \alpha_{\text{cross}}, \beta)$ apply to both types. $\alpha_{\text{self}} > \alpha_{\text{cross}}$ describes persistence of direction, as found in the crypto data of this work; $\alpha_{\text{cross}} > \alpha_{\text{self}}$ describes reversion, as at the equity fixed point of Lee (2023).

2.5 Stability

The mass matrix of the kernels is

$$\mathbf{G} = \frac{1}{\beta}\begin{bmatrix} \alpha_{\text{self}} & \alpha_{\text{cross}} \\ \alpha_{\text{cross}} & \alpha_{\text{self}} \end{bmatrix}.$$

Its entry (i, j) is the mean number of type-i children of a type-j event, so $\mathbf{G}$ plays the role of the branching ratio. Its eigenvalues are $(\alpha_{\text{self}} - \alpha_{\text{cross}})/\beta$ and the largest,

$$g = \frac{\alpha_{\text{self}} + \alpha_{\text{cross}}}{\beta},$$

which is the mean number of children of either type per event. The two eigenvalues have eigenvectors $(1, 1)$ and $(1, -1)$: the first governs the total activity $N_1 + N_2$, the second the difference $N_1 - N_2$, that is, the price (Section 2.7). Write

$$g_- = \frac{\alpha_{\text{self}} - \alpha_{\text{cross}}}{\beta}$$

for the second eigenvalue. The condition $g < 1$ is required. Close to one, cascades last longer; above one, there is no usual stationary regime (Hardiman; Bercot; Bouchaud, 2013; Filimonov; Sornette, 2012). Under stationarity, by the same argument as in the univariate case, each type occurs at mean rate $\mu/(1 - g)$, and the two together at $2\mu/(1 - g)$.

2.6 Marks and price

An event has a time, a type and a mark. The mark is the size of the change in ticks. The paper keeps the mark for volatility, but assumes that the intensity does not depend on it. The filtered price can be written, in ticks, as the cumulative difference between up-moves and down-moves.

2.7 Volatility

In a minimal model, if the price moves by one tick ϑ at a rate ϕ,

$$\sigma^2 \approx \phi\vartheta^2.$$

Thus, modeling intensity is modeling activity and, consequently, a component of volatility. The marked Hawkes expression requires moments of the marks and the estimated parameters (Lee; Seo, 2017; Lee, 2024).

Long-run variance of the Hawkes price. With unit marks the price is $X(t) = N_1(t) - N_2(t)$, the projection on the eigenvector $(1, -1)$. Under stationarity each type has mean rate $\bar{\lambda} = \mu/(1 - g)$, and the variance of X per unit time converges, as the horizon grows, to

$$\lim_{T\to\infty} \frac{\operatorname{Var} X(T)}{T} = \frac{2\bar{\lambda}}{(1 - g_-)^2}$$

(Bacry; Delattre; Hoffmann; Muzy, 2013). If each event moves the price by an i.i.d. size J, independent of the times, with the sign given by the type, the same argument gives

$$\sigma_H^2 = 2\bar{\lambda}\left[\operatorname{Var}(J) + \frac{\mathbb{E}[J]^2}{(1 - g_-)^2}\right], \tag{2.5}$$

the expression used in Section 6.10. The total activity depends on g, and the amplification of the variance on g_-. When self-excitation dominates, $g_- > 0$ and moves in the same direction cluster, so the variance exceeds that of a Poisson price with the same activity. When cross-excitation dominates, $g_- < 0$, moves tend to undo each other and the variance falls below it; this is the mechanism that Bacry, Delattre, Hoffmann & Muzy (2013) use to model microstructure noise. When $\alpha_{\text{self}} = \alpha_{\text{cross}}$, the long-run variance equals that of the difference of two independent Poisson processes with the same activity.

Realized volatility. The benchmark is the realized variance over a window,

$$\mathrm{RV}_\Delta = \sum_k r_k^2,$$

the sum of squared log returns sampled every Δ seconds (Andersen; Bollerslev; Diebold; Labys, 2003; Barndorff-Nielsen; Shephard, 2002). Without noise, it converges to the integrated variance as $\Delta \to 0$. At high frequency, however, the observed price contains microstructure noise, and RV_Δ is biased upward when Δ is small (Zhang; Mykland; Aït-Sahalia, 2005). The signature plot, RV_Δ per unit time against Δ, shows the effect: noise makes it fall with Δ, while positively autocorrelated returns, as in a self-exciting price, make it rise toward the long-run level of Equation (2.5).

2.8 Simulation

A Hawkes path can be simulated by Ogata's thinning (Ogata, 1981). If $\lambda(t) \leq M$ on an interval, candidates are drawn from a Poisson process of rate M and each candidate at time s is kept with probability $\lambda(s)/M$; the kept points have intensity λ. With an exponential kernel the intensity only decreases between events, so its value just after the last event is a valid bound until the next event. In the bivariate case the bound is $\lambda_1 + \lambda_2$, and the type of an accepted event is drawn in proportion to λ_1 and λ_2. The implementation is described in Section 5.3.

2.9 Maximum likelihood

For a horizon T,

$$\ell(\theta) = \sum_k \log \lambda_{c_k}(t_k) - \sum_i \int_0^T \lambda_i(u)\, du.$$

The first term rewards high intensity at the events; the second penalizes high intensity over the whole interval. Maximizing only the first would drive the intensities to infinity. The compensator provides the balance. For an exponential kernel, the sum over the past obeys a first-order recursion, and ℓ and its gradient can be computed in $O(n)$ operations (Ozaki, 1979). In the univariate case, let $A_1 = 0$ and $A_k = e^{-\beta(t_k - t_{k-1})}(1 + A_{k-1})$, so that αA_k is the excitation just before t_k (Equation (2.2)). Then

$$\ell(\mu, \alpha, \beta) = \sum_{k=1}^{n} \log\big(\mu + \alpha A_k\big) - \mu T - \frac{\alpha}{\beta} \sum_{k=1}^{n} \Big(1 - e^{-\beta(T - t_k)}\Big). \tag{2.6}$$

The bivariate likelihood has the same structure, with one recursion per pair of source and target types. Under regularity conditions, the MLE is consistent and asymptotically efficient (Ogata, 1978): approximately normal around the true value, with covariance given by the inverse Fisher information, which the inverse of the observed Hessian of $-\ell$ estimates. These conditions include a true parameter in the interior of the parameter space. When it lies on the boundary, as $\alpha_{\text{cross}} = 0$, the estimate piles up at the boundary with positive probability, and neither the normal approximation nor the usual χ^2 reference for likelihood ratio tests applies (Self; Liang, 1987). This is relevant for the real data of this work, where α_{cross} is estimated close to zero.

2.10 Model comparison

The Poisson process is nested in the Hawkes process: it is the case $\alpha_{\text{self}} = \alpha_{\text{cross}} = 0$. The likelihood ratio statistic $2(\hat{\ell}_{\text{Hawkes}} - \hat{\ell}_{\text{Poisson}})$ measures the gain in fit. Its usual χ^2 reference does not hold here for two reasons: the null value lies on the boundary of the parameter space (Self; Liang, 1987), and β has no effect under the null, so it is not identified there. The Akaike information criterion,

$$\text{AIC} = 2k - 2\hat{\ell},$$

with k the number of free parameters, compares the two fits with a penalty for the extra parameters (Akaike, 1974). A difference in AIC of hundreds of units, as found on the real windows, leaves no doubt about the direction, whatever the exact reference distribution.

2.11 Goodness of fit by time rescaling

If the model is correct, the compensator turns the observed process into a unit-rate Poisson process: the transformed times $\Lambda(t_k)$ are the events of a homogeneous Poisson process of rate one. This is the time-rescaling theorem (Brown et al., 2002). The increments

$$\tau_k = \Lambda(t_k) - \Lambda(t_{k-1}) = \int_{t_{k-1}}^{t_k} \lambda(u)\,\mathrm{d}u$$

are then independent $\text{Exp}(1)$ variables. In the bivariate case the theorem applies to each type with its own compensator Λ_i. Evaluated at the estimated parameters, the τ_k are the rescaled residuals of the fit (Ogata, 1988). Their distribution is tested against $\text{Exp}(1)$, for example with Kolmogorov–Smirnov, and their serial dependence with Ljung–Box. A kernel of the wrong shape typically leaves dependence in the residuals: bursts that the fitted kernel forgets too fast or too slowly appear as runs of short or long τ_k.

2.12 Observation on a time grid

The events of this work are changes of the mid-price recorded on a 100 ms grid (Chapter 4). Two things are lost relative to continuous time: event times are rounded to the grid, and changes that fall in the same cell merge into one. The second matters most for the decay. With β of a few units per second, much of the excitation acts within a few hundred milliseconds, and merged

events remove part of the short inter-arrivals from which β is identified. A likelihood written for continuous time therefore estimates the parameters of the process as seen through the grid, with β biased downward; Section 6.12.2 measures the size of this bias. Likelihoods that account for the observation scheme have been proposed for this setting (Lambe; Chen; Stindl; Kwan, 2026).

A biased estimator can still be corrected by simulation, the idea of indirect inference (Gouriéroux; Monfort; Renault, 1993). Let $b(\theta)$ be a summary, such as the median, of the grid MLE over paths simulated from θ in continuous time and mapped to the grid. The map b is estimated by simulation, and an estimate on the grid is corrected by solving $b(\theta) = \hat{\theta}_{\text{grid}}$. The correction is as good as the model used to simulate: if the real process is not a Hawkes process, the map transfers its bias with it. A network trained on gridded paths with the continuous-time parameter as target learns a correction of the same kind, conditional on the whole path.

2.13 Identifiability in finite samples

The asymptotic properties do not guarantee good behavior with 1,000 events. When the excitation is weak (small $\alpha_{\text{self}} + \alpha_{\text{cross}}$), the kernel barely changes the intensity, and the likelihood becomes flat in the direction of β: very different values of β explain the data almost equally well. In the limit $\alpha \to 0$, β is no longer identified. Near criticality, the separation between μ and g also deteriorates, because the mean rate $2\mu/(1-g)$ is well determined, but neither factor is on its own. These effects reappear in the results of Chapter 6.

2.14 Amortized estimation as a Bayes estimator

In the approach of Lee (2023), a network f_ϕ is trained on pairs (θ_i, x_i), where θ_i is drawn from a training distribution π and x_i is simulated from the model with parameter θ_i. Minimizing the mean squared error,

$$\min_{\phi} \; \mathbb{E}_{\theta\sim\pi}\, \mathbb{E}_{x|\theta} \left\| f_\phi(x) - \theta \right\|^2,$$

is minimizing the Bayes risk under squared loss, whose solution is the posterior mean $\mathbb{E}_\pi[\theta \mid x]$ (Lehmann; Casella, 1998). Hence, a well-trained network approximates a Bayes estimator under the prior π (Sainsbury-Dale; Zammit-Mangion; Huser, 2024; Lenzi; Bessac; Rudi; Stein, 2023). Two consequences guide this work. First, on a test set drawn from the same π, no estimator has lower average MSE than the posterior mean, so outperforming the MLE on that test is expected and does not prove superiority in a frequentist sense. Second, the network does not extrapolate:

its estimates shrink toward the high-density region of π, and real data outside the support of the prior yield biased estimates.

The same approach belongs to a wider family of amortized, simulation-based inference methods, in which the cost of inference is paid once, in training, and each new data set costs a forward pass (Cranmer; Brehmer; Louppe, 2020; Radev et al., 2022). On real windows the target changes. When a network is trained on pairs $(x, \hat{\theta}_{\mathrm{MLE}}(x))$, the label is a deterministic function of the input, so the minimizer of the squared loss is the MLE itself: the network becomes a fast approximation of the MLE map, accurate where the training windows are dense. Starting from the weights trained on simulated data, it can reach that approximation with fewer real windows; how close the simulation prior is to the real data then matters for how much that initialization helps.

2.15 Comparing estimators

The estimators are compared path by path. For each path or window i, a loss $L_i(A)$, such as the squared error averaged over the four parameters, is computed for each estimator A; the macro MSE is the average of the per-parameter MSEs. Two estimators are compared through the paired differences $d_i = L_i(A) - L_i(B)$. Pairing removes the variation in difficulty between paths, which is often much larger than the difference between estimators. The mean $\bar{d}$ is reported with a bootstrap confidence interval, obtained by resampling the d_i with replacement (Efron; Tibshirani, 1993), and the null hypothesis of no difference is tested with the Wilcoxon signed-rank test, which uses the ranks of $|d_i|$ and is robust to the heavy tails of squared errors (Wilcoxon, 1945; Diebold; Mariano, 1995). Variation across training seeds is a different source: it measures the instability of training, not the accuracy of a trained model, and is compared across seeds (Bouthillier et al., 2021).

When several comparisons are tested together, the Holm procedure controls the probability of any false rejection: with the m p-values sorted in increasing order, the k-th is compared with $0.05/(m - k + 1)$, and testing stops at the first non-rejection (Holm, 1979). Agreement between two orderings, such as the Hawkes and the realized volatilities across windows, is measured by the Spearman correlation, the correlation of the ranks, which ignores the level and depends only on the order.

Ensembles. The average $\bar{y} = \frac{1}{M}\sum_m y_m$ of M estimators satisfies

$$\|\bar{y} - \theta\|^2 = \frac{1}{M}\sum_{m=1}^{M}\|y_m - \theta\|^2 - \frac{1}{M}\sum_{m=1}^{M}\|y_m - \bar{y}\|^2$$

(Krogh; Vedelsby, 1995). The error of the average equals the average error of the members minus their spread, so it is never worse than the average member, and it improves most when the members are accurate and err in different places. This identity applies both to the ensembles of five seeds and to the average of the LSTM and the Transformer.

2.16 LSTM

Recurrent networks update a state $h_t = f(x_t, h_{t-1})$. The LSTM introduces input, forget and output gates to control the flow of information and mitigate the vanishing gradient problem (Hochreiter; Schmidhuber, 1997). With σ the logistic function and $\odot$ the elementwise product,

$$\begin{aligned} f_t &= \sigma(W_f[h_{t-1}, x_t] + b_f), & i_t &= \sigma(W_i[h_{t-1}, x_t] + b_i), \\ o_t &= \sigma(W_o[h_{t-1}, x_t] + b_o), & \tilde{c}_t &= \tanh(W_c[h_{t-1}, x_t] + b_c), \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, & h_t &= o_t \odot \tanh(c_t). \end{aligned}$$

The cell state c_t is updated additively, which lets gradients pass over many steps. The update has the form of Equation (2.2): a forget gate that depends on the inter-arrival time can play the role of $e^{-\beta\Delta_k}$, and the input gate that of the jump, so an LSTM can represent the excitation of a Hawkes process in its state. In Lee, the sequence contains the inter-arrival time and the type; the final state feeds the regression of the four parameters. The paper also tested a GRU (Cho; Merriënboer; Bahdanau; Bengio, 2014), with slightly worse results.

2.17 Attention and the Transformer

Attention computes weighted combinations:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V.$$

Each position queries all the others. The Transformer combines multiple heads, feed-forward layers, residual connections and normalization (Vaswani et al., 2017). Each head applies its own

projections $Q = XW^Q$, $K = XW^K$, $V = XW^V$ to the sequence X, and the heads are concatenated. Self-attention is equivariant to permutations of the sequence, so order and spacing must enter through a positional encoding; for a point process the relevant position is the time of each event, not only its index. The potential advantage is capturing distant dependencies; the potential cost is $O(n^2)$ memory. The position of the normalization relative to the residual connections affects training stability: the *post-LN* variant, used here, usually requires learning-rate warm-up (Xiong et al., 2020).

2.18 Training

The networks are trained by stochastic gradient descent on mini-batches with Adam, which scales each step by running estimates of the first and second moments of the gradient (Kingma; Ba, 2015). The learning rate can follow a schedule, such as a short warm-up or a cosine decay (Xiong et al., 2020; Loshchilov; Hutter, 2017). Training stops at a fixed budget of epochs or when the validation loss stops improving, and the checkpoint with the lowest validation loss is kept; the test data are not used for any choice. Fine-tuning reuses this procedure from weights already trained on another task, here the simulated paths, with the real windows as the new training set.

3 Related Work

3.1 Hawkes processes in finance

After the original formulation (Hawkes, 1971b; Hawkes, 1971a), self- and mutually exciting models came to describe market events in continuous time (Bowsher, 2007; Embrechts; Liniger; Lin, 2011), contagion between markets (Aït-Sahalia; Cacho-Diaz; Laeven, 2015) and endogeneity, measured by the branching ratio (Filimonov; Sornette, 2012; Hardiman; Bercot; Bouchaud, 2013). Bacry, Delattre, Hoffmann & Muzy (2013) use a bivariate Hawkes process of up- and down-moves to reproduce the microstructure noise of the price; cross-excitation captures reversion at short scales. Bacry, Mastromatteo & Muzy (2015) and Hawkes (2018) review this field, and Toke (2011) offers a didactic introduction. Lee & Seo (2017) connect the marked Hawkes process to the price in ticks and derive Hawkes volatility, which Lee (2024) apply to filtered prices. In Brazil, Primo (2018) analyzes the intraday clustering of stocks with MLE, and Silva (2026) systematizes inference, diagnostics and financial applications.

3.2 Estimation of Hawkes processes

MLE for exponential kernels dates back to Ozaki (1979), with asymptotic properties established by Ogata (1978). Simulation by *thinning* (Ogata, 1981) and diagnostics by rescaled residuals (Ogata, 1988; Brown et al., 2002) complete the classical toolbox (Laub; Lee; Taimre, 2021). The optimization is sensitive to the initialization and to the geometry of the likelihood, which can be nearly flat in some directions when the series is short (Section 2.13).

3.3 Amortized and simulation-based inference

When the likelihood is expensive or intractable, networks trained on simulations replace the optimization with a single forward pass (Cranmer; Brehmer; Louppe, 2020). Networks trained on simulations already estimate parameters of spatial and extreme-value models (Gerber; Nychka, 2021; Lenzi; Bessac; Rudi; Stein, 2023); with squared loss, they approximate Bayes estimators (Sainsbury-Dale; Zammit-Mangion; Huser, 2024), and methods such as BayesFlow learn the full posterior (Radev et al., 2022). Lee (2023) applies the idea to the 2D Hawkes process with an

LSTM. Lambe, Chen, Stindl & Kwan (2026) use networks for the Hawkes process observed only through interval counts, a case in which the exact likelihood is intractable and the MLE is no longer a direct alternative.

3.4 Neural point processes

Another line of work uses networks to model the intensity itself, without the parametric form of Hawkes: RMTPP (Du et al., 2016), the Neural Hawkes Process (Mei; Eisner, 2017) and attention-based variants such as the Transformer Hawkes Process (Zuo et al., 2020) and the Self-Attentive Hawkes Process (Zhang; Lipani; Kirnap; Yilmaz, 2020); Shchur, Türkmen, Januschowski & Günnemann (2021) review the area. The goal differs from that of this work: those models maximize predictive likelihood, whereas here the network estimates the four interpretable parameters of a classical model.

3.5 Hawkes processes in crypto-assets

Using one-minute returns of 17 cryptocurrencies traded on Binance, Taddeo, Salvi & Rivera-Castro (2024) show that Hawkes describes the occurrence of jumps in log-returns better than Poisson. The crypto market also exhibits segmentation and limited arbitrage across exchanges (Makarov; Schoar, 2020), and perpetual futures, which have no expiry and carry a funding rate, concentrate a large share of the liquidity (He; Manela; Ross; Wachter, 2022).

3.6 Positioning

The present work combines controlled reproduction, public crypto data and a comparison with attention, and adds to the evaluation of Lee (2023) robust metrics, paired tests and the interpretation of the network as a Bayes estimator.

Table 3.1 – Positioning of this work relative to the closely related literature.

Work	Estimation	Architecture	Market
Bacry, Delattre, Hoffmann & Muzy (2013)	MLE	–	futures (Bund, DAX)
Lee & Seo (2017)	MLE	–	equities
Primo (2018)	MLE	–	equities (Brazil)
Taddeo, Salvi & Rivera-Castro (2024)	MLE	–	crypto (jumps in 1-min returns)
Lee (2023)	network	LSTM	equities (US)
Lambe, Chen, Stindl & Kwan (2026)	network	MLP	synthetic (counts)
Zuo et al. (2020)	neural intensity	Transformer	generic
This work	network and MLE	LSTM and Transformer	crypto (perpetual futures)

4 Data

4.1 Source and market

Both spot and futures were evaluated. The need for the best bid/ask ruled out the official spot archive, which has no equivalent historical bookTicker. USD-M futures provide bookTicker and aggTrades. The symbols are BTCUSDT and ETHUSDT (Binance, 2026).

4.2 Schema

bookTicker includes an identifier, the best bid and ask prices and quantities, the transaction time and the event time. The canonical schema has a UTC timestamp, bid, ask and quantities. Audits check for positive prices, bid less than or equal to ask, ordering and temporal duplicates.

4.3 Sampling

Updates can occur every millisecond, a much finer resolution than the chosen scale. Timestamps are grouped into 100 ms bins and the last state is kept. This filter follows Lee's protocol and reduces ultra-high-frequency noise.

4.4 Datasets produced

Table 4.1 – Dataset coverage.

Window	Symbol-days	Events	Segments of 1,000
Primary (Jan. 2024)	28	1,826,904	1,813
Robustness (Oct. 2023)	14	1,288,352	1,281

4.5 Event characteristics

Table 4.2 summarizes the events of the calibration block (January 15 to 22, 2024). Three facts distinguish these data from the setting of Lee (2023). First, about 76% of the changes repeat

the direction of the previous one, which indicates persistence rather than reversion. Second, the median jump is 10 to 15 ticks and only 10–13% of the jumps are one tick: at the 100 ms scale, these contracts behave as small-tick assets. Third, 37–39% of the inter-arrival times are exactly 0.1 s, the grid step; the sampling strongly discretizes the times, whereas the model and the simulations assume continuous time.

Table 4.2 – Characteristics of the mid-price events, January 15 to 22, 2024.

	BTCUSDT	ETHUSDT
Events	529,278	540,164
Fraction of up-moves	49.4%	49.5%
Probability of repeating the previous direction	76.0%	75.7%
Median inter-arrival time	0.3 s	0.3 s
Inter-arrival times equal to 0.1 s (grid)	39.5%	37.3%
Median / 90th percentile jump (ticks)	15.5 / 48	10 / 29
One-tick jumps	9.7%	12.6%

5 Methodology

5.1 Principles

The protocol distinguishes:

R1: explicitly published information;

R2: reconstruction to fill a gap;

A1: adaptation required for crypto;

E1: extension with a Transformer.

5.2 Pipeline

1. build the daily URL and download the ZIP;
2. compute the SHA-256 and record it in a manifest;
3. temporarily extract the CSV;
4. process chunks of one million rows;
5. reduce the updates to the 100 ms grid;
6. produce Parquet partitioned by symbol and date;
7. delete the temporary CSV;
8. create windows without crossing splits;
9. fit the MLE and calibrate ranges on the training set only;
10. generate synthetic paths and train the estimators;
11. select hyperparameters on the validation set;
12. open the test set only once.

5.3 Simulation by thinning

Given the current state, an upper bound on the sum of the intensities is computed. An exponential waiting time is drawn; the excitation decays; the candidate is accepted with probability proportional to the new intensity (Ogata, 1981). The type is drawn with probabilities proportional to λ_1, λ_2. After the event, the corresponding column of the α matrix is added. Each path starts with an empty history, the same assumption as in the likelihood.

5.4 Constrained MLE

Positive parameters are obtained through softplus. Instead of optimizing β freely, a positive margin d is used:

$$(\mu, \alpha_s, \alpha_c, d) = \text{softplus}(z), \qquad \beta = \alpha_s + \alpha_c + d.$$

L-BFGS-B is started from five points: one based on the empirical event rate and four random perturbations in the unconstrained space. The MLE never receives the true parameter as initialization. The gradient is analytic, and the exponential recursions are solved as bidiagonal systems. Convergence, iterations, log-likelihood and time are recorded.

5.5 Neural protocol

The target is $(\mu, \alpha_s, \alpha_c, \beta)$, and the loss is the MSE on the original scale. The input for each event is $(\log(1 + \Delta t),\ s)$, with $s = +1$ for an up-move and $s = -1$ for a down-move; Lee uses inter-arrival times and types 1/2, so this transformation is a reconstruction (R2). The two networks share the output head: softplus produces $\mu, \alpha_s, \alpha_c, d > 0$ and $\beta = \alpha_s + \alpha_c + d$, which guarantees $g < 1$ by construction, as in Lee's robust version.

The frozen synthetic experiment contains 75,000 training sequences, 10,000 validation sequences and 15,000 test sequences, all of exactly 1,000 events. The parameters are drawn with $\mu \in [0.2, 0.6]$, $\beta \in [0.8, 2.0]$, self-excited fraction $\alpha_s/(\alpha_s + \alpha_c) \in [0.2, 0.8]$ and g balanced across three strata: low $[0.10, 0.35)$, intermediate $[0.35, 0.70)$ and near-critical $[0.70, 0.95]$. Table 5.1 summarizes the training.

Table 5.1 – Training configuration and cost (five seeds per network, Apple Silicon Mac with MPS).

	LSTM (R1/R2)	Transformer (E1)
Architecture	2 LSTM × 12 + dense	encoder, 2 layers, $d = 32$
Heads / feed-forward	–	4 / 128
Trainable parameters	2,068	25,700
Optimizer	Adam, 10^{-3}	Adam, 10^{-3}
Effective batch	64	8 (physical 4 with accumulation)
Maximum epochs / patience	300 / 30	300 / 30
Best epoch (per seed)	165, 164, 193, 239, 299	73, 42, 38, 60, 44
Training time per seed	5.4–14.8 h	16.6–28.8 h
Total time (5 seeds)	41 h	110 h

The Transformer's tensor flow starts from $[B, L, 2]$, with $L = 1000$. A linear projection produces $[B, L, 32]$, to which the positional encoding is added. In each layer, the four heads operate in subspaces of dimension eight; the feed-forward block expands 32 to 128 and returns to 32. Temporal averaging converts $[B, L, 32]$ into $[B, 32]$. The memory cost of attention grows quadratically with L, which is why its batch is smaller than the LSTM's. This difference in batch size, together with the absence of a hyperparameter search for either network, is a limitation of the comparison.

5.6 The paper's fixed point (E2)

To compare directly with the sampling-distribution table of Lee (2023), 2,000 paths of 1,000 events were simulated at the published point $(\mu, \alpha_s, \alpha_c, \beta) = (0.3, 0.4, 0.7, 1.5)$, which lies inside the training prior. The networks were applied without retraining, and the MLE was fitted in two versions: the protocol's heuristic and the initialization at the true value used in the paper.

5.7 Empirical protocol

On real data there is no true parameter, so the empirical stage asks three narrower questions: whether the Hawkes model describes the windows well enough to be estimated, whether a network can reproduce the MLE on real windows, and whether the estimated parameters give a volatility consistent with the realized one. The protocol was fixed in a decision record (ADR 008) before the test block was opened. The windows have 1,000 events and are split chronologically by day: calibration (15–22 January 2024, 1,063 windows), validation (23–25 January, 476 windows) and test (26–28 January, 274 windows); the October 2023 week is a robustness window that is never used for training or selection.

Goodness of fit. With the MLE of each calibration window, the time-rescaled residuals of each dimension are tested against Exp(1) with Kolmogorov–Smirnov and for serial dependence with Ljung–Box at 10 lags (Ogata, 1988; Brown et al., 2002), and the Hawkes model is compared with a homogeneous bivariate Poisson process by likelihood ratio and AIC. Since the times lie on a 100 ms grid, two controls separate the effect of the grid from misspecification: times jittered uniformly inside their cell with the MLE refitted, and paths simulated from each fitted model, either in continuous time or mapped to the grid in the same way as the data. The rejection rate of the simulated controls is what a correct model would produce.

Networks on real windows. Each window is labeled with its MLE (five heuristic starts). Both architectures are trained on the calibration labels with the loss, batch size, maximum of 300 epochs and patience of 30 of the synthetic protocol, in two variants: from scratch (Adam, 10^{-3}) and fine-tuned from the synthetic weights of the same seed (Adam, 10^{-4}), each with the five protocol seeds. Two references need no training: the synthetic networks as they are, and the mean of the training labels, which is the best constant predictor. The selected variant is the one whose five-seed ensemble has the lowest macro MSE on validation. The test block is labeled and read once, after the selection; the stored weights are evaluated without retraining, and every variant is reported. The machine budget for this stage was capped at 30 hours.

Volatility. The long-run volatility implied by the estimated parameters (Section 6.10) is compared window by window with the realized volatility of the log mid-price sampled every $\Delta \in \{0.1, 1, 5, 10, 30\}$ seconds, using the median ratio and the Spearman correlation across windows, with $\Delta = 10$ s as the main horizon, for the MLE and for the selected network.

5.8 Metrics and tests

The reported metrics are per-parameter and aggregate MSE, median absolute error, bias, validity, branching-ratio error, cost and MLE convergence. Because the parameters have different scales, the R^2 of each parameter relative to the prior mean (the trivial estimator) is also reported. The comparison between networks uses two sources of variation: across seeds (Welch and Mann–Whitney tests with five seeds), which measures training variability (Bouthillier et al., 2021), and across test paths (paired difference with a *bootstrap* CI and Wilcoxon test), which measures the accuracy of the already trained models (Efron; Tibshirani, 1993; Diebold; Mariano, 1995).

6 Results

6.1 Data gate

All 56 primary files were present. Twenty-eight partitions were generated. BTCUSDT produced 887,803 events; ETHUSDT, 939,101. The independent window produced 1,288,352 events. The availability and volume gate was therefore satisfied.

6.2 Reproduction at the fixed point

Table 6.1 – Sampling distribution at the fixed point of Lee (2023) ($\mu = 0.3$, $\alpha_{\text{self}} = 0.4$, $\alpha_{\text{cross}} = 0.7$, $\beta = 1.5$): mean and standard deviation over 2,000 paths of 1,000 events. Lee's rows use 100,000 paths of 2,000 steps.

	μ		α_{self}		α_{cross}		β	
Estimator	Mean	SD	Mean	SD	Mean	SD	Mean	SD
LSTM (5-seed ensemble)	0.3044	0.0365	0.4950	0.0536	0.6245	0.0608	1.5318	0.1436
Transformer (5-seed ensemble)	0.2957	0.0388	0.5280	0.0561	0.6055	0.0620	1.5308	0.1541
Heuristic MLE (5 starts)	0.3065	0.0370	0.3956	0.0728	0.7075	0.0859	1.5156	0.1642
Oracle MLE (started at the true value)	0.3065	0.0370	0.3956	0.0728	0.7075	0.0859	1.5156	0.1642
Lee (2023) network	0.3151	0.0358	0.4429	0.0661	0.5733	0.0697	1.5736	0.1364
Lee (2023) MLE	0.3036	0.0314	0.3988	0.0500	0.7024	0.0608	1.5078	0.1145

Table 6.1 reproduces the qualitative pattern of Lee (2023). The MLE is practically unbiased, and its standard deviations are about $\sqrt{2}$ times those in the paper, as expected for paths of half the length. The networks show the characteristic shrinkage bias: they pull α_{self} and α_{cross} toward the center of the prior, as in the paper, in exchange for lower variance. The two versions of the MLE produced exactly the same estimates, so the initialization at the true value used by Lee does not explain the MLE's advantage at this well-identified point ($g \approx 0.73$). The macro MSE was 0.0103 for the MLE and 0.0111 for the LSTM ensemble.

6.3 Final comparison: LSTM, Transformer and MLE

Results of the final optimized run

The final protocol uses only paths of 1,000 events, with 75,000 training samples, 10,000 validation samples and 15,000 test samples. The networks are summarized over five independent seeds; the

MLE is fitted separately on each path of the same test set. The network intervals use Student's t distribution across seeds.

Table 6.2 – Aggregate metrics on the frozen synthetic test set (15,000 paths of 1,000 events).

Estimator	Macro MSE	95% CI	Macro MAE	MSE of g	Validity
LSTM	0.019618	[0.013252, 0.025984]	0.094276	0.001825	100.00%
Transformer	0.023700	[0.019128, 0.028272]	0.105642	0.002138	100.00%
MLE	0.506193	–	0.111852	0.002344	100.00%

Table 6.3 – Test MSE by parameter (networks: mean over five seeds).

Estimator	μ	α_{self}	α_{cross}	β
LSTM	0.002296	0.016222	0.016566	0.043386
Transformer	0.002366	0.020905	0.021196	0.050332
MLE	0.010108	0.006146	0.006214	2.002303

Table 6.4 – Macro MSE by regime of the branching ratio g.

Estimator	Low g	Intermediate g	Near-critical g
LSTM	0.019941	0.015614	0.023297
Transformer	0.022228	0.018964	0.029908
MLE	1.480568	0.016795	0.021214

Between the networks, the LSTM obtained a mean macro MSE 17.2% lower than the Transformer. With 5 and 5 seeds, however, the difference is not statistically significant (Welch test, $p = 0.19$), and the confidence intervals overlap; the data do not allow the networks to be ranked. For the MLE, the parameter β accounted for 98.9% of the sum of squared errors, and the macro MSE in the low-g regime was 88.15 times that of the intermediate regime: with weak excitation, β is weakly identified and some paths produce extreme estimates. Outside this regime the MLE is competitive: in the intermediate regime its macro MSE was 0.0168 (LSTM: 0.0156; Transformer: 0.0190) and, in the near-critical regime, 0.0212 (LSTM: 0.0233; Transformer: 0.0299). The MLE obtained the lowest MSE among the three estimators for α_{self} and α_{cross}. Therefore, the aggregate macro MSE does not rank the estimators robustly, and numerical convergence does not imply good parameter recovery.

The MLE converged on 100.00% of the paths and used a median of 14.00 iterations. All comparisons use the same frozen test set.

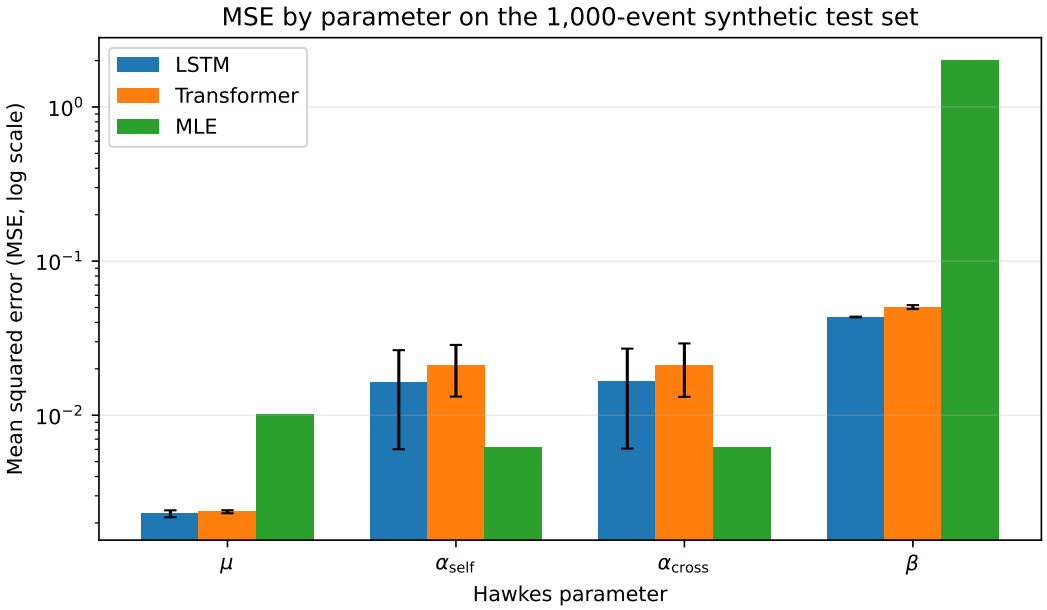


Figure 6.1 – Test MSE by parameter under the 1,000-event protocol. The network error bars represent one standard deviation across seeds.

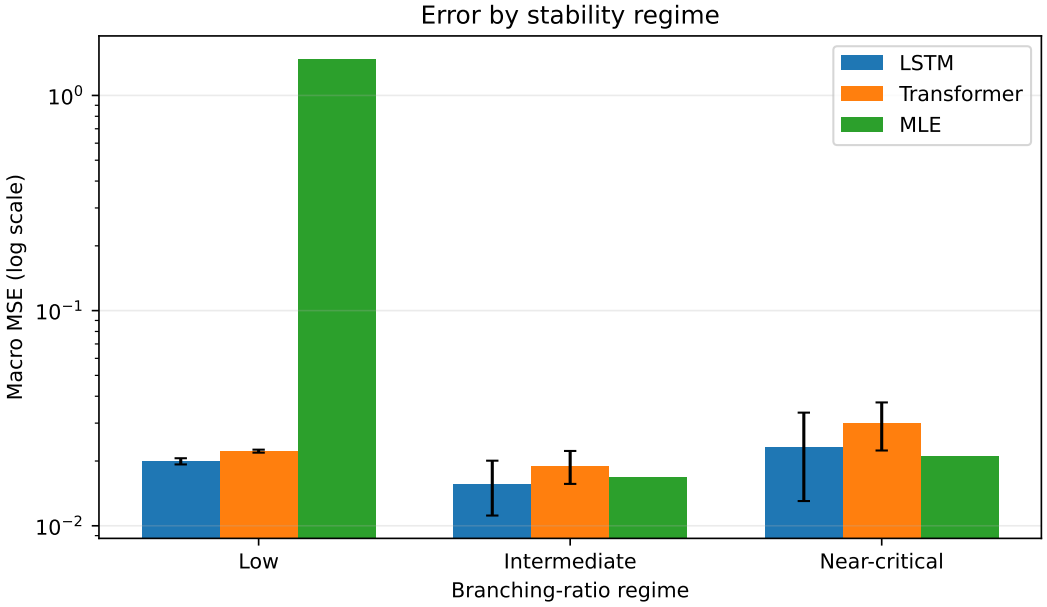


Figure 6.2 – Macro MSE by regime of the branching ratio. The network error bars represent one standard deviation across seeds.

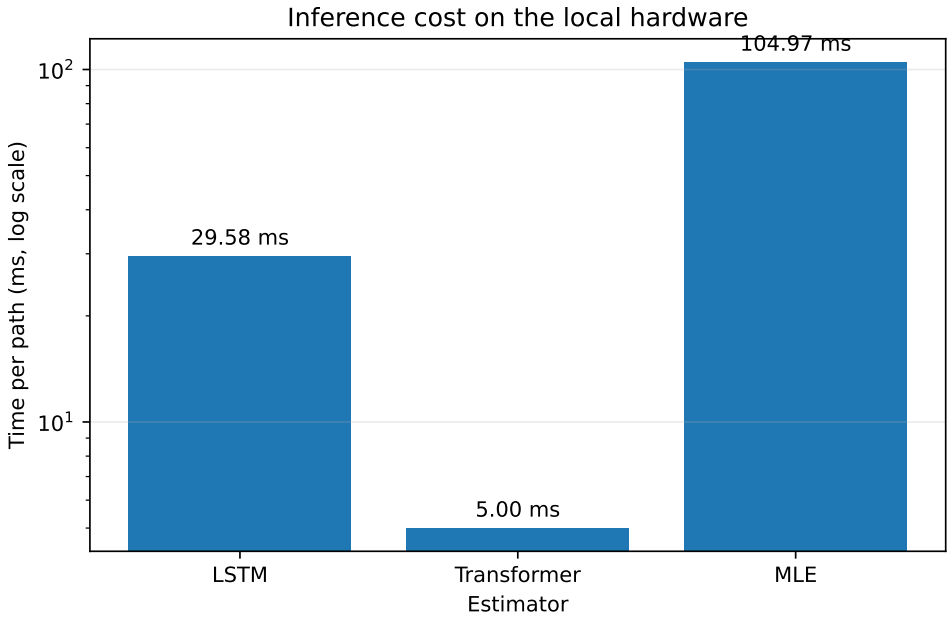


Figure 6.3 – Cost per path recorded during the final run, on a logarithmic scale. For the MLE, wall-clock time with four processes competing with training on the GPU; for the networks, median latency at batch size one on MPS. A later controlled benchmark measured about 10 ms per path for the MLE on a single core and should be preferred to this figure.

6.4 Complementary analysis of the synthetic results

The per-path predictions were recomputed from the frozen checkpoints (maximum relative difference of 10^{-8} with respect to the reported MSE), without retraining and without changing the protocol. Because the parameters have different scales, each squared error is divided by the variance of the parameter in the training set; the complementary result is the R^2 relative to the trivial estimator that always predicts the prior mean.

Table 6.5 – Per-parameter R^2 relative to the prior mean, macro median absolute error and MSE of the branching ratio on the synthetic test set.

Estimator	μ	α_{self}	α_{cross}	β	Median AE	MSE of g
Prior mean	−0.01	0.00	−0.01	0.01	0.1908	0.0670
MLE	0.24	0.90	0.90	−15.64	0.0674	0.0023
LSTM (ensemble)	0.84	0.82	0.81	0.65	0.0680	0.0017
Transformer (ensemble)	0.84	0.74	0.73	0.61	0.0765	0.0019

First, the networks' advantage over the MLE in aggregate MSE comes from a few paths: 94.4% of the sum of squared errors of the MLE's β is concentrated in 1% of the paths, almost all of them in the low-g regime. In all of the 300 worst paths, the log-likelihood at the estimate exceeds that at the true parameter (median gain of 2.80), and refitting 100 of them with 20 starts barely

Table 6.6 – Paired comparison by path (mean standardized squared error over the four parameters). Columns: median loss of A and of B, fraction of paths in which A has the smaller error, and *bootstrap* 95% CI of the mean difference A−B (negative favors A).

A × B	Regime	Med. A	Med. B	A better	95% CI (A−B)
LSTM × Transformer	all	0.151	0.187	70.2%	[−0.051, −0.047]
LSTM × Transformer	low g	0.112	0.131	61.1%	[−0.021, −0.016]
LSTM × Transformer	intermediate g	0.127	0.164	72.8%	[−0.045, −0.041]
LSTM × Transformer	near-critical g	0.237	0.304	76.7%	[−0.089, −0.081]
LSTM × MLE	all	0.151	0.149	48.8%	[−9.394, −0.809]
LSTM × MLE	low g	0.112	0.199	63.5%	[−26.977, −2.225]
LSTM × MLE	intermediate g	0.127	0.100	40.1%	[−0.040, −0.009]
LSTM × MLE	near-critical g	0.237	0.183	42.7%	[−0.467, −0.277]
Transformer × MLE	all	0.187	0.149	43.5%	[−9.376, −0.782]
Transformer × MLE	low g	0.131	0.199	59.4%	[−27.686, −2.207]
Transformer × MLE	intermediate g	0.164	0.100	34.2%	[0.002, 0.035]
Transformer × MLE	near-critical g	0.304	0.183	36.7%	[−0.374, −0.195]

changes the result. This is not a numerical failure: with weak excitation, β is weakly identified in 1,000 events. Second, on the typical path the MLE is as good as the networks or better: its median loss is practically equal to the LSTM's overall and lower in the intermediate and near-critical regimes. Third, in the paired comparison the LSTM ensemble beats the Transformer ensemble in every regime; across individual seeds, however, the difference is not significant (Welch, $p = 0.19$; Mann–Whitney, $p = 0.056$), because the LSTM showed two levels of convergence across seeds.

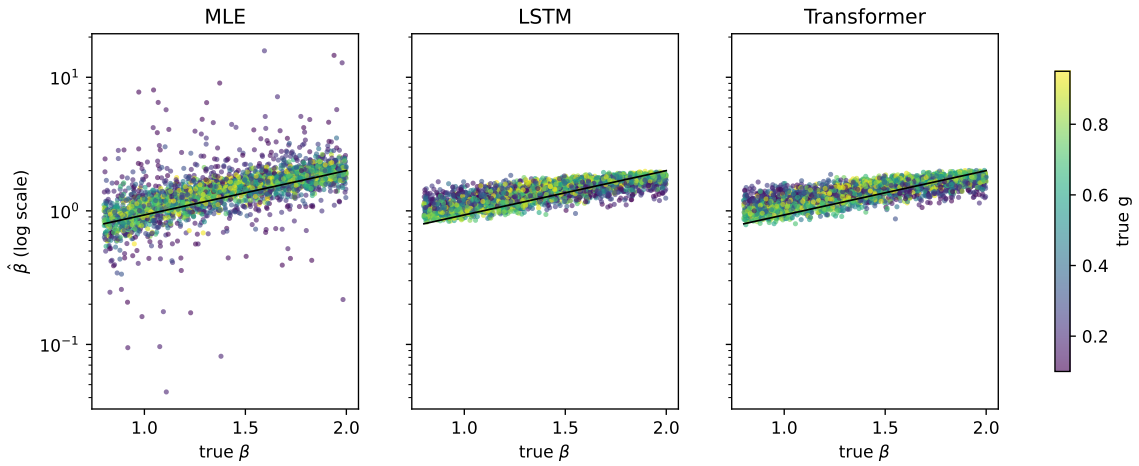


Figure 6.4 – True versus estimated β (3,000 test paths, logarithmic scale). The networks shrink the estimates toward the center of the prior; the MLE is unbiased over most of the domain but produces extreme values when g is low.

6.5 Empirical calibration and support of the synthetic prior

The MLE was fitted on the 1,063 windows of 1,000 events in the `calibration_train` block (15 to 22 January 2024), the only block authorized for calibration. The median β was 2.71 (interquartile range 2.21–3.14), above the upper limit of 2.0 used in training, and α_{cross} was below 10^{-3} in 89.4% of the windows. Only 15.4% of the windows have β inside the prior, 0.4% have the self-excited fraction inside $[0.2, 0.8]$, and 0.0% satisfy all the ranges at once. Because an estimator trained with quadratic loss approximates the posterior mean under the training prior, the current networks cannot produce estimates in this region; the empirical evaluation requires recalibrating the prior or training on MLE labels from real windows.

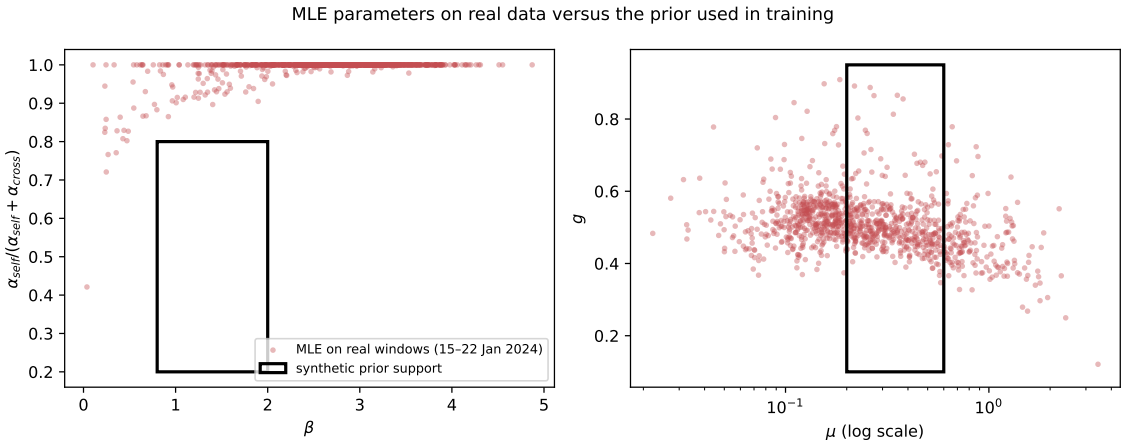


Figure 6.5 – MLE estimates on the real calibration windows (points) and support of the prior used to generate the training data (rectangles).

6.6 Computational cost

The final run recorded 105 ms of wall-clock time per path for the MLE with four processes. This value was measured while neural training was occupying the GPU. A controlled benchmark on the same machine, with the same code (which exactly reproduces the 15,000 recorded estimates), produced the values in Table 6.7.

With an analytic gradient and a linear recursion, the MLE of this four-parameter model costs a few milliseconds, the same order as neural inference at batch size one. The ratio of about 150 times reported by Lee is not reproduced. The cost advantage of the networks appears in batched inference, especially for the LSTM, and would grow for models with more expensive likelihoods.

Table 6.7 – Cost per path of 1,000 events in a controlled benchmark.

Estimator and execution mode	Time per path
MLE, 5 starts, one core	9.6 ms
MLE, 1 start, one core	2.2 ms
MLE, 5 starts, 4 processes (wall-clock)	2.5 ms
LSTM, batch size one, CPU (1 thread) / MPS	7.0 ms / 29.6 ms
Transformer, batch size one, CPU (1 thread / 10 threads) / MPS	9.9 / 4.8 / 5.0 ms
LSTM, batch of 256 on MPS	0.07 ms
Transformer, batch of 32 on MPS	3.7 ms
Lee (2023): network / MLE	12 ms / 1,763 ms

Lee does not describe the implementation of the MLE. To find where its 1,763 ms come from, the same paths at Lee's fixed point were fitted with implementations that differ in one factor at a time, at 1,000 and 2,000 events, the two lengths the paper mentions (Appendix A). All variants reach the same estimates (largest difference 0.002 in any parameter); only the cost changes (Table 6.8). The likelihood is the dominant factor: computed as a double sum over pairs of events, it costs about 90 times as much as the recursion at 2,000 events, while a numerical gradient and a derivative-free optimizer (Nelder–Mead) multiply the cost by about five each. With the recursion, even an interpreted loop stays below 0.1 s, and the time reported by Lee falls between the two vectorized double-sum variants at 2,000 events. The reported ratio is therefore most likely due to a likelihood computed without the recursion of Section 2.9.

Table 6.8 – Cost of one MLE fit by implementation, at the fixed point of Lee (2023): median of 10 paths, one start at the true parameters, one core. The pure-Python double sum was timed for one evaluation and multiplied by the number of evaluations of the vectorized Nelder–Mead fit.

Likelihood	Gradient and optimizer	1,000 events	2,000 events
Recursion, vectorized	analytic, L-BFGS-B (this work)	1.1 ms	1.7 ms
Recursion, vectorized	numerical, L-BFGS-B	5.4 ms	8.3 ms
Recursion, Python loop	numerical, L-BFGS-B	47 ms	89 ms
Double sum, vectorized	numerical, L-BFGS-B	176 ms	711 ms
Double sum, vectorized	Nelder–Mead	818 ms	3,358 ms
Double sum, Python loop	Nelder–Mead (extrapolated)	26 s	105 s
Lee (2023), implementation not described			1,763 ms

6.7 Empirical calibration

The MLE converged on all 1,063 calibration windows. The details are in Section 6.5: cross-excitation is practically zero, β exceeds the upper limit of the prior in most windows, and no window falls inside all the ranges used in training. The following sections report the empirical protocol of Section 5.7.

6.8 Goodness of fit on real data

The diagnostic uses only the calibration block (1,063 windows of 1,000 events, 15–22 January 2024), with the MLE already fitted to each window. Under a correctly specified model, the time-rescaled residuals of each dimension are i.i.d. Exp(1) (Ogata, 1988; Brown et al., 2002); they are tested with Kolmogorov–Smirnov (KS) against Exp(1) and with Ljung–Box (10 lags) for serial dependence. Because the data are observed on a 100 ms grid, two controls separate grid effects from misspecification: real times jittered uniformly within their cell (with the MLE refitted), and paths simulated from each window's fitted Hawkes, either kept in continuous time or mapped to the grid (one event per cell) and jittered in the same way.

Table 6.9 – Residual diagnostics per window and dimension. Under a correct model, about 5% of the tests reject at the 5% level.

Model and data	KS rejects	Median KS p	Ljung–Box rejects
Poisson, real data	100.0%	$< 10^{-47}$	40.6%
Hawkes, real data	100.0%	$< 10^{-8}$	46.2%
Hawkes, real data, jittered grid	70.3%	0.021	47.1%
Hawkes, simulated on the grid, jittered	47.7%	0.054	4.7%
Hawkes, simulated in continuous time	0.9%	0.626	4.9%

The Hawkes model is far better than a homogeneous Poisson process: the likelihood-ratio test rejects Poisson at 1% in 99.8% of the windows, and the median AIC difference is -721 in favour of Hawkes. It is not, however, the data-generating process. The grid alone makes KS reject in 47.7% of the simulated controls, so the 70.3% rejection on real data is only partly misspecification. Ljung–Box, on the other hand, keeps its nominal size on the grid (4.7%) and rejects in 47.1% of the real windows: part of the clustering is left in the residuals, which a single exponential kernel cannot absorb. Jittering barely moves the MLE (median β from 2.71 to 2.77; median α_{self} from 1.34 to 1.36). On real data, therefore, the target of a neural estimator is the

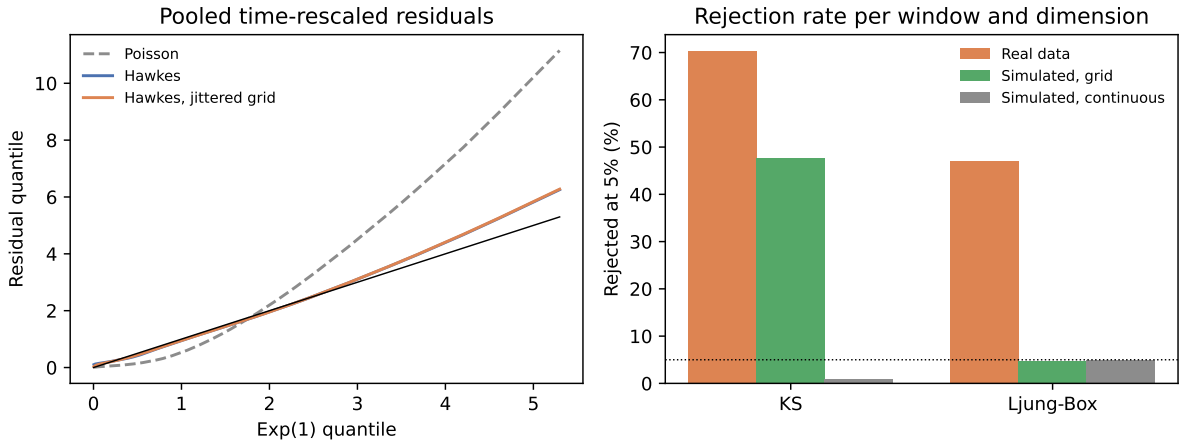


Figure 6.6 – Left: Q–Q plot of the pooled time-rescaled residuals against Exp(1). Right: rejection rates on real data and in the two simulated controls; the dotted line is the nominal 5%.

MLE of this approximating model, not a true parameter.

The same diagnostic was repeated once on the 274 test windows (26–28 January 2024), after the test block was opened. The pattern is the same: the likelihood ratio rejects Poisson in 100.0% of the windows (median AIC difference −905), and Ljung–Box rejects in 47.1% of the jittered real windows, against 4.2% in the simulated controls on the grid.

6.9 Neural estimators on real windows

Each real window is labelled with its MLE (five heuristic starts), and the networks learn to reproduce that label. Training uses the calibration block (1,063 windows), selection uses the validation block (476 windows, 23–25 January 2024). Each variant is trained with the 5 seeds of the synthetic protocol, either from scratch or starting from the synthetic weights of the same seed (fine-tuning, learning rate divided by 10); the synthetic networks without retraining and the mean of the training labels are reference points. The selected variant is the one with the lowest validation macro MSE of the 5-seed ensemble. The test block was opened once, after the selection was frozen.

6.10 Hawkes volatility

For the symmetric model with exponential kernels, kernel norms $a_s = \alpha_{\text{self}}/\beta$ and $a_c = \alpha_{\text{cross}}/\beta$, stationary intensity $\Lambda = \mu/(1 - a_s - a_c)$ per direction and i.i.d. jump sizes J (absolute log

Table 6.10 – Agreement with the MLE label on real windows: macro MSE of the seed ensemble and mean ± standard deviation across seeds on the validation block, and R^2 per parameter relative to the training-label mean (share of the label variance reproduced). *Selected variant.

Variant	Ensemble	Seeds	R^2_μ	$R^2_{\alpha_{\text{self}}}$	R^2_β	Test
Training-label mean	0.2096	–	0.00	0.00	0.00	0.2184
LSTM, synthetic weights	0.5550	0.5669 ± 0.0793	0.52	−4.48	−1.19	0.2501
Transformer, synthetic weights	0.5575	0.5707 ± 0.0161	0.59	−4.63	−1.08	0.2747
LSTM, trained from scratch	0.1385	0.1483 ± 0.0098	0.61	0.39	0.30	0.2502
LSTM, fine-tuned *	0.0359	0.0454 ± 0.0133	0.90	0.85	0.82	0.0442
Transformer, trained from scratch	0.1002	0.1060 ± 0.0012	0.93	0.50	0.47	0.1692
Transformer, fine-tuned	0.0413	0.0522 ± 0.0071	0.94	0.78	0.79	0.0396

returns), the long-run variance of the price per unit time is

$$\sigma_H^2 = 2\Lambda \left[\mathrm{Var}(J) + \frac{\mathbb{E}[J]^2}{(1 - a_s + a_c)^2} \right] \tag{6.1}$$

(Bacry; Delattre; Hoffmann; Muzy, 2013). Each window's Hawkes volatility, σ_H times the square root of its duration, is compared with the realized volatility of the log mid-price sampled every Δ seconds (Andersen; Bollerslev; Diebold; Labys, 2003). The metrics were fixed before the test was opened: the median ratio σ_H/σ_{RV} and the Spearman correlation across windows, with $\Delta = 10$ s as the main horizon. The estimate is finite in 100.0% of the windows, and its median absolute log change between consecutive windows (0.097) is close to that of the realized volatility (0.102).

Table 6.11 – Hawkes volatility against realized volatility: median ratio σ_H/σ_{RV} and Spearman correlation across windows, by sampling interval Δ.

		$\Delta = 1$ s		$\Delta = 10$ s		$\Delta = 30$ s	
Block	Parameters	Ratio	ρ_S	Ratio	ρ_S	Ratio	ρ_S
Calibration	MLE	1.37	0.87	1.27	0.74	1.30	0.59
Calibration	network	1.39	0.56	1.30	0.40	1.34	0.36
Validation	MLE	1.36	0.79	1.24	0.67	1.27	0.51
Validation	network	1.37	0.59	1.24	0.47	1.28	0.37
Test	MLE	1.42	0.86	1.27	0.78	1.30	0.60
Test	network	1.56	0.34	1.38	0.28	1.45	0.33

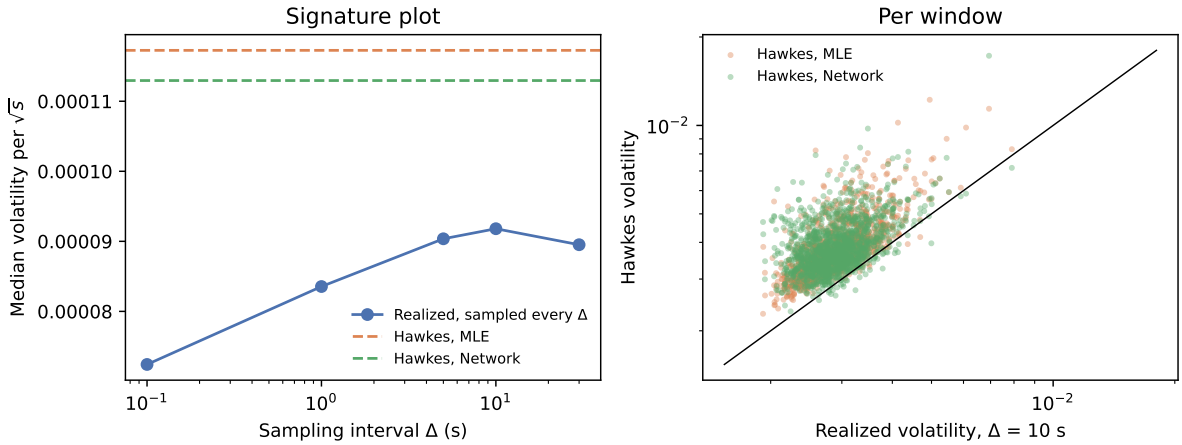


Figure 6.7 – Left: median realized volatility per $\sqrt{s}$ as a function of the sampling interval (signature plot) and the median Hawkes level. Right: Hawkes against realized volatility ($\Delta = 10$ s) per window.

6.11 Robustness window

The week of 23–29 October 2023 was never used for training or selection. It is cut into 1,281 non-overlapping windows of 1,000 events, labeled with the MLE (convergence in 100.0% of the windows), and every variant is evaluated with the weights stored before the test was opened.

Table 6.12 – Agreement with the MLE label in the October 2023 robustness window: macro MSE of the seed ensemble, overall and by tercile of realized volatility. *Variant selected on the January validation block.

Variant	All	Low vol.	Medium vol.	High vol.
Training-label mean	0.6292	0.4997	0.3329	1.0550
LSTM, synthetic weights	1.1153	1.2838	1.2134	0.8487
Transformer, synthetic weights	1.0596	1.2621	1.1169	0.7997
LSTM, trained from scratch	0.4087	0.4052	0.3289	0.4920
LSTM, fine-tuned *	0.1212	0.1076	0.0712	0.1849
Transformer, trained from scratch	0.3329	0.4486	0.3120	0.2383
Transformer, fine-tuned	0.0828	0.0343	0.0386	0.1757

The two fine-tuned networks were also compared window by window, as a post hoc analysis that was not part of the pre-registered protocol: the selection had already been made on validation. The loss of each window is the squared error averaged over the four parameters, and the difference is LSTM minus Transformer. On the test block the difference is 0.0046, with a paired bootstrap 95% interval of $[-0.0020, 0.0112]$ and Wilcoxon $p = 0.385$: the two variants cannot be told apart. In the October window the difference is 0.0384 ($[0.0319, 0.0449]$, $p < 10^{-32}$), and the Transformer has the lower loss in 64.3% of the windows: under a change of

regime, the fine-tuned Transformer degrades less than the selected LSTM.

6.12 Addendum: analyses registered after the results were frozen

This addendum was registered (ADRs 009 to 011) after the main results had been frozen and does not change them: the model selected in the main protocol remains the fine-tuned LSTM of Section 5.7. It tests the hypothesis raised by October, measures the bias of the 100 ms grid, evaluates a better fine-tuning recipe and a reduced version of scope B on weeks that played no role in any decision, and revisits the volatility criterion.

6.12.1 Does the LSTM degrade faster when β moves?

The October result suggested that the LSTM stores the decay in its weights, tuned to the January β, and degrades faster than the Transformer when β changes. To test this without new market data, 400 paths per value of β were simulated with $\mu = 0.23$, $g = 0.5$ and $\alpha_c = 0.001$ at the centre of the January labels, taken to the 100 ms grid exactly as the real data and cut to 1,000 events. The target is the MLE of each path, which is what the fine-tuned networks learned to imitate; the frozen weights are used without retraining (Table 6.13).

Table 6.13 – Mean loss against the MLE on simulated paths, ensemble of five seeds. The first row is the true β of the simulation; * marks values added after the registered grid.

True β	1.2	2.0	2.7	3.5	4.5	6.0*	8.0*
Median MLE β on the grid	0.93	1.45	1.86	2.28	2.73	3.33	3.96
LSTM, fine-tuned	0.017	0.007	0.022	0.028	0.028	0.037	0.125
Transformer, fine-tuned	0.029	0.097	0.114	0.091	0.053	0.038	0.059

By the registered rule the hypothesis is supported: the LSTM-minus-Transformer difference grows by 0.067 from $\beta = 2.7$ to $\beta = 4.5$, with a 95% bootstrap interval of [0.061, 0.073]. The premise of the rule, however, failed. The grid merges events that fall in the same 100 ms cell and pulls the MLE β down: a true β of 4.5 yields a median label of 2.73, the centre of January, not a shift away from it. In the space of the labels, the LSTM has the lower loss inside and below the January range, where it is the Transformer that degrades; the two tie near the October median ($\beta \approx 3.2$); and above the training range (95% of the January labels lie below 3.74) the loss of the LSTM grows about fourfold and crosses that of the Transformer. The hypothesis therefore holds only for β above the training range, and β alone does not explain the

October gap, since October also has larger μ and α_s. A side result: on the grid, the MLE recovers between 50% and 80% of the continuous-time β in this range.

6.12.2 How much of β does the 100 ms grid hide?

The β sweep showed that the grid pulls the MLE β down. This post hoc analysis measures the effect over the region of the real labels. For each pair (μ, β) of a mesh, 100 paths with $g = 0.5$ and $\alpha_c = 0.001$ were simulated in continuous time, taken to the grid (one event per cell), cut to 1,000 events and fitted by the MLE (Table 6.14). The median of the MLE defines a map from continuous-time parameters to grid labels, interpolated in logarithms and inverted for each real window, which is a form of indirect inference.

Table 6.14 – Median MLE β on the 100 ms grid for each continuous-time (μ, β), $g = 0.5$, 100 paths per cell.

μ \ true β	1.2	2.0	2.7	3.5	4.5	6.0	8.0	10.0	13.0
0.05	1.04	1.63	2.11	2.62	3.17	3.83	4.63	5.23	5.97
0.1	1.01	1.54	2.01	2.46	3.00	3.71	4.38	5.01	5.65
0.23	0.93	1.45	1.88	2.27	2.72	3.32	4.01	4.48	5.02
0.5	0.81	1.26	1.63	1.97	2.38	2.84	3.40	3.82	4.28
1.0	0.63	0.96	1.26	1.51	1.83	2.21	2.65	2.96	3.25
2.0	0.42	0.61	0.86	0.98	1.14	1.48	1.66	1.88	1.98

The bias grows with both β and μ, that is, with the share of events that fall in an occupied cell: the larger the intensity relative to the 100 ms step, the more events are merged and the slower the decay looks. The branching ratio is also biased down, from 0.5 to between 0.22 and 0.48. Inverting the map for the January labels (inside the map in 95.4% of the training windows), the median β of 2.71 on the grid corresponds to 4.7 in continuous time (5%–95%: 1.9–9.2), and the grid label is a median 58.3% of the continuous value; on validation the figures are 4.9 and 56.5%. The October median of 3.19 corresponds to 6.5. The estimate of μ is nearly unaffected. First, the labels of this work describe the process as seen through the grid, not the process in continuous time; the real decay is roughly twice as fast, about 213 ms of mean memory instead of 369 ms. Second, jittering the times inside their cell, used in the diagnostics, does not undo this bias, because it cannot restore the merged events; it only tests the effect of ties. The inversion assumes $g = 0.5$ and $\alpha_c \approx 0$ and treats the median as the target; it gives the order of magnitude of the bias, not a corrected estimate per window.

6.12.3 A better fine-tuning recipe

The fine-tuned LSTM reached its best epoch between 291 and 299 of 300, so the original run stopped before convergence, and both fine-tuned networks predicted α_c worse than the mean of the labels because β dominated the loss. Five new arms start from the same synthetic weights, with the same data, seeds, learning rate and batch size: a 1,000-epoch cap; the cap plus a five-epoch warm-up, halving of the learning rate after ten epochs without improvement and gradient clipping; and the latter plus a training loss standardized by the variance of the training labels (floored at 0.01). Checkpoints and early stopping use the validation MSE on the original scale, as in the main protocol, and the selection rule is the same (Table 6.15). With the longer cap the LSTM stopped at epochs 158, 308, 441, 836, 925.

Table 6.15 – Validation macro MSE of the five-seed ensembles. The selection among the new arms was recorded before the new week was read.

Variant	Validation MSE
LSTM, fine-tuned (frozen, selected in the TG)	0.0359
Transformer, fine-tuned (frozen)	0.0413
LSTM, 1,000-epoch cap	0.0294
LSTM, cap + schedule	0.0358
Transformer, cap + schedule	0.0448
LSTM, cap + schedule + standardized loss	0.0354
Transformer, cap + schedule + standardized loss	0.0431
Average of the best LSTM and the best Transformer	0.0304

Selected on validation: the LSTM with the 1,000-epoch cap.

6.12.4 A new week as a clean test

The test block of the main protocol had already been opened, so the new arms were evaluated on a week that nobody had seen: BTCUSDT and ETHUSDT, 5–11 February 2024, processed exactly as the primary period[1] and cut into 711 non-overlapping windows of 1,000 events labelled by the MLE. The week was read once, after the selection (Table 6.16).

Primary comparison, registered in advance: the LSTM with the 1,000-epoch cap minus the frozen LSTM, window by window. The mean difference is −0.0052 (paired bootstrap 95% interval [−0.0060, −0.0044], Wilcoxon $p < 10^{-38}$), and the new arm has the lower loss in 73.3%

[1]The Binance files of that week have their lines interleaved out of time order; each file was sorted by transaction time and update identifier, which agree after sorting, before the same processing.

Table 6.16 – Macro MSE and R^2 per parameter on the new week, and macro MSE on October 2023 (exploratory), ensembles of five seeds. R^2 of α_c is left out of the table because its label variance is close to zero, which makes it unstable; it is discussed in the text.

Variant	MSE	R^2_μ	$R^2_{\alpha_s}$	R^2_β	October
LSTM, fine-tuned (frozen, selected in the TG)	0.0264	0.79	0.80	0.78	0.1212
Transformer, fine-tuned (frozen)	0.0285	0.91	0.78	0.75	0.0828
LSTM, 1,000-epoch cap	0.0212	0.86	0.83	0.82	0.1158
LSTM, cap + schedule	0.0260	0.79	0.82	0.79	0.1310
Transformer, cap + schedule	0.0295	0.92	0.79	0.74	0.0942
LSTM, cap + schedule + standardized loss	0.0315	0.79	0.84	0.71	0.1017
Transformer, cap + schedule + standardized loss	0.0330	0.90	0.81	0.69	0.0854
Average of the best LSTM and the best Transformer	0.0204	0.91	0.85	0.82	0.0868
Mean of the training labels	0.1826	−0.82	−0.11	−0.62	0.6292

of the windows. The secondary comparisons, with Holm-adjusted p-values, are in Table 6.17.

Table 6.17 – Secondary paired comparisons on the new week (difference A minus B; negative favours A). The interval is for the mean difference and the p-value is Wilcoxon's, which looks at the median, so the two can disagree when a few windows dominate the mean.

A vs B	Difference	95% interval	Holm p
LSTM, schedule vs LSTM (frozen)	−0.0004	[−0.0009, 0.0001]	0.613
Transformer, schedule vs LSTM (frozen)	0.0031	[0.0004, 0.0060]	0.559
LSTM, standardized vs LSTM (frozen)	0.0051	[0.0037, 0.0067]	$< 10^{-7}$
Transformer, standardized vs LSTM (frozen)	0.0067	[0.0041, 0.0092]	$< 10^{-5}$
LSTM + Transformer average vs LSTM (frozen)	−0.0059	[−0.0075, −0.0042]	$< 10^{-12}$
LSTM, schedule vs Transformer, schedule	−0.0035	[−0.0063, −0.0008]	0.559
LSTM, standardized vs Transformer, standardized	−0.0016	[−0.0041, 0.0009]	0.613

The registered primary hypothesis is supported: training the same LSTM for longer lowers the loss on a week that played no role in any decision, by about a fifth, and the gain is consistent window by window. The two best candidates on validation are the two best on the new week, in swapped order. The schedule (warm-up, plateau decay and clipping) did not help: with it the LSTM stopped earlier and tied with the frozen network. The standardized loss did what it was meant to do for α_c (R^2 from −21.1 to −1.0 for the LSTM and 0.05 for the Transformer, the only variants that are not clearly worse than the constant mean on this parameter), but paid for it in β and lost overall. The average of the LSTM and the Transformer, registered before the week was read and only second on validation, has the lowest loss on the new week (0.0204), which is consistent with the β sweep: the two networks err in different regions, so their errors partly cancel. No Transformer arm beat the frozen LSTM on the new week. In October, which had

been seen before and is exploratory, the picture of the main protocol remains: the Transformers have the lowest losses (0.083 frozen), the longer LSTM improves only slightly on the frozen one (0.116 against 0.121), and the average (0.087) is close to the best. Inside the regime of the training data, then, the LSTM remains the better single network and was undertrained in the main protocol; under a change of regime the Transformer errs less; and the average of the two has the lowest loss on the new week and is close to the best in October.

6.12.5 A synthetic prior built from the real labels (reduced scope B)

The main protocol pretrained the networks on a synthetic prior fixed in advance (scope A) and left for later work a prior built from the real data (scope B). A reduced version was registered (ADR 010) before any path was generated. The prior is a mixture: in 75% of the draws, the label of a random training window is inverted to continuous time by the map of Section 6.12.2, with log-normal noise on μ and β and the branching ratio inflated by a factor between 1 and 1.25 to offset the bias of the grid; in 25%, the parameters are drawn from a wide box around the real range. Each path is simulated in continuous time, taken to the 100 ms grid, cut to 1,000 events and labelled by the MLE on the grid, so that the synthetic target means the same as the real one. 22,000 paths were kept (none dropped), and the real labels fall inside the 1–99% synthetic range in 98.2% to 99.8% of the training windows, depending on the parameter. The LSTM was pretrained on these paths (five seeds, 150-epoch cap, which all seeds reached) and then fine-tuned on the real training windows with the recipe of Section 6.12.3 (best epochs 291, 320, 347, 628, 648). Since the February week had been read, a second unseen week was downloaded and read once: BTCUSDT and ETHUSDT, 4–10 March 2024, 1,860 windows (Table 6.18).

Table 6.18 – Macro MSE of the five-seed ensembles. Validation was recorded before the March week was read; February and October had been read before and are exploratory.

Variant	Validation	March 2024	February 2024	October 2023
Mean of the training labels	–	0.5815	0.1826	0.6292
LSTM, fine-tuned (frozen, selected in the TG)	0.0359	0.1169	0.0264	0.1212
LSTM, 1,000-epoch cap	0.0294	0.1085	0.0212	0.1158
Transformer, fine-tuned (frozen)	0.0413	0.1121	0.0285	0.0828
Average of the two above	0.0304	0.1020	0.0204	0.0868
LSTM, scope B prior, no fine-tuning	0.1439	0.1220	0.2115	0.1414
LSTM, scope B prior, fine-tuned	0.0202	0.0403	0.0153	0.0336
Average of the latter and the Transformer	0.0196	0.0560	0.0137	0.0406

Primary comparison, registered in advance: the fine-tuned scope B LSTM minus the

LSTM with the 1,000-epoch cap, window by window on the March week. The mean difference is −0.068 (95% interval [−0.082, −0.056], Wilcoxon $p < 10^{-10}$), the loss falls by 62.9%, and the scope B network has the lower loss in 71.8% of the windows. The three secondary comparisons also favour it after Holm's correction: against the frozen LSTM, −0.077 ([−0.090, −0.064]); the average with the Transformer against the average of Section 6.12.4, −0.046 ([−0.053, −0.039]); and the pretrained network without fine-tuning against the mean of the labels, −0.460 ([−0.508, −0.414]); all with Holm $p < 10^{-10}$. The sign is the same on February and October.

The gain is concentrated where the earlier networks failed. March and October are the weeks farthest from the calibration period (the constant mean errs three times more there than on February), and most of the error of the earlier LSTM comes from μ: on March its MSE for μ is 0.309 ($R^2 = 0.61$), against 0.086 ($R^2 = 0.89$) for the scope B network, which also lowers the error in β from 0.097 to 0.064. The wide part of the prior covers activity levels that the January calibration does not, and the fine-tuning keeps the network anchored to the real labels: without it, the scope B network is no better than the earlier ones on March and much worse on February, so the level of the estimates comes from the fine-tuning. Unlike in Section 6.12.4, averaging with the Transformer does not help on March or October: once the LSTM extrapolates well, the Transformer mostly adds its own error. Scope B, reduced as it is, gives the largest out-of-sample gain of this work, and the selected model of the main protocol is unchanged by rule.

6.12.6 Can a network undo the bias of the grid?

The MLE on the grid cannot recover the continuous-time β (Section 6.12.2), but a network trained on gridded paths with the true parameters as target can learn the correction, which is a structural advantage of amortized estimation under discretized observation. The protocol was registered (ADR 011) before training. The LSTM was trained from scratch on the paths of scope B (Section 6.12.5) with their continuous-time parameters as target (five seeds, 150-epoch cap) and compared on 2,000 new paths of the same prior with two alternatives: the MLE on the grid and the MLE corrected by inverting the map of Section 6.12.2 (Table 6.19).

Primary comparison, registered in advance: network minus corrected MLE, path by path, mean difference −0.326 (95% interval [−0.405, −0.256], Wilcoxon $p < 10^{-10}$); the network has the lower loss in 60.5% of the paths. Secondary, with Holm: network minus MLE on the grid, −3.624 ([−3.912, −3.356], $p < 10^{-10}$); corrected minus uncorrected MLE, −3.297 ([−3.531, −3.065], $p < 10^{-10}$). The correction by the map alone removes almost all of the bias in β

Table 6.19 – Error against the continuous-time parameters on 2,000 new simulated paths. Relative error of β: signed and absolute medians.

Estimator	Macro MSE	MSE μ	MSE β	Bias β	|Error| β
LSTM trained on the continuous-time truth	0.328	0.0346	0.899	+2.8%	8.9%
MLE on the grid	3.951	0.0115	11.680	−44.1%	44.1%
MLE on the grid, corrected by the map	0.654	0.0108	1.589	−2.4%	8.4%

(median relative error from −44.1% to −2.4%) and has the lowest median error in β and in μ. The network wins on the mean through the tails of α_s and β, where the map, measured at a single branching ratio, extrapolates poorly.

On real windows the trained network implies a median continuous-time β of 7.7 on validation and 6.3 on February, against 4.9 and 3.1 from the inversion of the map and 2.76 and 2.16 on the grid (Spearman between network and inversion 0.55 and 0.55). On real data the two corrections disagree: the network places β well above the map. The real windows are not Hawkes paths (Section 6.8), so a correction learned on simulated paths does not transfer safely, and without the continuous-time truth neither can be checked. The network is the better correction inside the prior, but on market data it has not been verified. Training and test share the prior, which favours the network; the map was measured only at $g = 0.5$ and $\alpha_c \approx 0$, which penalizes the corrected MLE away from it. Both caveats were registered before the run.

6.12.7 Hawkes volatility with the addendum networks

Criterion C4 was measured with the selected network only, which ranks windows by volatility much worse than the MLE (Section 6.10). The same formula and the same benchmark (realized volatility with $\Delta = 10$ s) were applied, post hoc but with the comparison registered before computing it (ADR 011), to the networks of the addendum, on the 711 windows of the February week and the 476 validation windows (Table 6.20).

The average of the two networks ranks the February windows better than the frozen LSTM by 0.155 in Spearman (bootstrap 95% interval [0.130, 0.181]), the registered comparison. The level is right for every estimator, with ratios between 1.2 and 1.4; the estimators differ in the ranking. Training the LSTM longer barely improves it, while the Transformer comes closest to the MLE. The LSTM has the lower parameter error, but the Transformer preserves more of the differences between windows, on which the ranking depends; the MLE remains the better input for this use.

Table 6.20 – Spearman correlation between the Hawkes volatility and the realized volatility across windows, and median ratio of the two.

	February 2024		Validation	
Estimator	Spearman	Ratio	Spearman	Ratio
MLE of the window	0.765	1.35	0.667	1.24
LSTM, fine-tuned (frozen, selected in the TG)	0.471	1.41	0.468	1.24
LSTM, 1,000-epoch cap	0.494	1.39	0.501	1.23
Transformer, fine-tuned (frozen)	0.688	1.31	0.608	1.20
Average of the two above	0.626	1.36	0.586	1.22

7 Discussion

7.1 Assessment of the hypotheses

H1 — supported as to structure and volume. The public data make it possible to reconstruct sequences with the same structure as Lee's, in sufficient volume. Their statistical content, however, differs from that of equities: persistent direction, multi-tick jumps and times discretized by the grid (Section 4.5).

H2 — partially supported. The LSTM's error is of the same order as the MLE's: at the fixed point, 0.0111 versus 0.0103; on the synthetic test, the median per-path loss is practically equal. The substantially lower latency was not confirmed at batch size one against an efficient MLE (Section 6.6); it holds for batched inference.

H3 — not supported on synthetic data. The Transformer did not reduce the error: its mean macro MSE was higher than the LSTM's, with no significance across seeds, and the Transformer ensemble lost to the LSTM ensemble in every regime. Nor did it perform better in the near-critical regime, where long memory would be most relevant. Its advantage was latency on MPS (5 ms versus 30 ms for the LSTM), at the cost of 2.7 times longer training. The result holds for the configuration tested; there was no hyperparameter search, and the *post-LN* Transformer was trained without learning-rate warm-up. On real windows, the fine-tuned architectures are indistinguishable on the test block, and in the October 2023 window the Transformer has a lower loss than the selected LSTM in 64% of the windows (Section 6.11). This robustness under a change of regime is the first evidence in favor of H3, but it comes from a post hoc comparison and not from the pre-registered selection, so it is a hypothesis for future work rather than a confirmation. The addendum (Section 6.12) sharpens it. On a new week, pre-registered and read once, no Transformer arm beat the frozen LSTM, while the average of the two networks had the lowest loss; in the β sweep, the LSTM errs less inside the training range and the Transformer above it. This complementarity depends on the prior: pretrained on a prior built from the real labels (Section 6.12.5), the LSTM alone is the best estimator on March and October, and averaging no longer helps. The Transformer keeps one advantage, for volatility: it preserves more of the ranking of the windows (Section 6.12.7).

7.2 Why the networks win on aggregate MSE

The MLE's macro MSE (0.506) is about 25 times that of the networks, but this difference stems from two mechanisms, not from greater accuracy of the networks on a typical path. The first is identifiability (Section 2.13): in the low-g regime, the MLE produces extreme values of β that legitimately maximize the likelihood, and 1% of the paths account for 94% of the squared error of β. The second is the prior (Section 2.14): the networks approximate the posterior mean under the training prior and never leave the interval $[0.8, 2.0]$ for β, which bounds their worst-case error. This robustness is useful, but it comes at the cost of shrinkage bias and depends on the prior representing the data.

7.3 Comparison with Lee (2023)

Lee's qualitative conclusions hold: the network reaches error of the same order as the MLE, with shrinkage bias, and the MLE is nearly unbiased when the parameter is well identified. The absolute MSE values are not comparable across the two works, since they depend on the parameter ranges, which the paper does not publish. The results diverge from the paper in two respects: the speed gain of two orders of magnitude is not reproduced with an efficient MLE (a likelihood computed by double sum over pairs of events recovers it; Section 6.6), and evaluating by aggregate MSE alone hides the identifiability failures that initialization at the true value tends to mask.

7.4 What the real data show

The model. In 99.8% of the calibration windows the Hawkes model is significantly better than a homogeneous Poisson process, so the clustering of price changes is real and large (Section 6.8). The model is, however, misspecified: the rescaled residuals keep serial dependence in about half of the windows, against 5% in simulated controls observed on the same grid. A single exponential kernel describes most of the clustering, and the 100 ms grid explains only part of the departure from $\mathrm{Exp}(1)$. On real data, therefore, the networks learn the MLE of an approximating model. It is also the MLE of the process seen through the grid: merging events in the same cell slows the apparent decay, and the grid label is about 58% of the continuous-time β (Section 6.12.2). A network trained on the true parameters of gridded paths undoes this bias in simulation, better on

average than inverting the map, but on real windows the two corrections disagree (Section 6.12.6).

The parameters. The estimated parameters ($\alpha_{\text{cross}} \approx 0$, $\beta \approx 2.7$) lie outside the support of the synthetic prior. The null cross-excitation contrasts with the typical point for equities, where it dominates (at Lee's fixed point, $\alpha_{\text{cross}} > \alpha_{\text{self}}$), and is consistent with the directional persistence measured in the data. Applied as they are, the synthetic networks shrink their estimates into the prior and are worse than the constant mean of the labels.

The networks. Trained on MLE labels of real windows, as Lee (2023) suggests, the networks become useful only when they start from the synthetic weights: from scratch, 1,063 windows are too few: on the test block the LSTM loses to the label mean and the Transformer beats it only modestly. Fine-tuned, both explain most of the label variance, with test macro MSE five times lower than the label mean. Synthetic training thus works as an informative initialization even when its prior does not cover the data. The LSTM chosen on validation and the fine-tuned Transformer cannot be told apart on the test block; in October the Transformer degrades less. The selected LSTM was also undertrained: with a 1,000-epoch cap and nothing else changed, its loss on a new week falls by a fifth (Section 6.12.4). The prior had a larger effect than the budget: pretrained on a prior built from the real labels and simulated on the same grid (a reduced scope B), the LSTM cuts the loss on a second new week by almost two thirds, mostly because it extrapolates μ to activity levels that the January calibration did not cover (Section 6.12.5).

The volatility. With the MLE, the Hawkes volatility ranks the windows like the realized volatility (Spearman 0.78 on the test block at $\Delta = 10$ s) and is about 1.27 times higher at horizons of 5 to 30 s. The ratio falls from about 1.6 at 0.1 s to 1.27 at 10 s, so the realized variance per unit time grows with Δ, as expected when successive price changes are positively autocorrelated, which is the directional persistence that the self-excitation describes. The Hawkes expression is a long-run quantity ($\Delta \to \infty$); since the ratio is still about 1.3 at 30 s, either the realized variance has not yet reached its long-run level or the model overstates the persistence. With the network, the level is kept but the ranking falls (0.28): the network shrinks its estimates toward the mean of the training labels, which compresses the differences between windows. For volatility, the estimator must also preserve the cross-section of the windows. The Transformer does this better than the LSTM (Spearman 0.69 against 0.47 on February, Section 6.12.7), but the MLE remains the best input (0.77).

7.5 Threats to validity

- **Internal:** leakage through overlapping windows; tuning after observing the test set. Mitigated by chronological splits and a test set locked behind a flag.
- **Construct:** on real data the label is the MLE of a model that the residual diagnostics reject in part, so the networks are evaluated on reproducing an approximating fit; α_{cross} lies on the boundary of the parameter space, where the usual asymptotic theory does not hold. The labels are also biased by the 100 ms grid (about 58% of the continuous-time β), which jittering inside the cell does not undo.
- **Prior:** the synthetic metrics depend on the sampled ranges, which were not calibrated on the data; the reduced scope B, which calibrates them, was run for the LSTM only, with a single prior design.
- **External:** two assets and two windows do not represent the whole market; the empirical test block covers three days, including a weekend.
- **Statistical:** five seeds quantify only part of the training variability; the LSTM showed two convergence levels across seeds, and one seed reached the 300-epoch limit while still improving. On real windows, the fine-tuned LSTM reached its best epoch at 291–299 in every seed, so it was also limited by the epoch budget; the addendum confirms that a longer budget lowers its loss on a new week. The comparison between fine-tuned networks on the test and October blocks is post hoc.
- **Reproduction:** the paper omits ranges, seeds and training details.
- **Computational:** latency depends on hardware, batch size, warm-up and contention between processes.

8 Future Work

The empirical stage of this work (scope A) answered whether networks can reproduce the MLE on real windows. The following steps are left for a master's continuation:

1. extend the prior built from the real labels (scope B), which the addendum ran in a reduced form for the LSTM only (Section 6.12.5), to the Transformer, to other prior designs and to more weeks;
2. estimate the continuous-time parameters under the 100 ms observation scheme: the addendum showed that a network trained on the true parameters of gridded paths undoes the grid bias in simulation (Section 6.12.6), but disagrees with the inverted map on real windows; a grid-aware likelihood (Lambe; Chen; Stindl; Kwan, 2026) and a prior that covers the misspecification of the real data are needed before the correction can be trusted on the market;
3. replace the single exponential kernel with sums of exponentials or power-law kernels, guided by the residual dependence found in the diagnostics;
4. make the estimators equivariant to the time scale, for example by normalizing the inter-arrival times of each window, and encode continuous time in the Transformer;
5. search hyperparameters and use pre-LN with a larger batch, without reusing any test block for tuning; the addendum already showed that a longer budget helps the LSTM, that warm-up with plateau decay and a standardized loss do not, and that averaging the LSTM and the Transformer is the best choice on a new week; the October robustness of the Transformer still needs a pre-registered week outside the training regime;
6. train for volatility directly, with a loss that preserves the ranking of the windows, and compare the Hawkes and realized volatilities at longer horizons;
7. extend the data to more assets, venues and periods.

9 Conclusion

This work built a reproducible pipeline that goes from Binance's public data to the comparison of three estimators under a frozen protocol, and it reproduced the qualitative pattern of Lee (2023) at the published fixed point. On the synthetic test, the LSTM did not lose to the Transformer, and attention brought no gain in accuracy. The networks' advantage over the MLE in aggregate MSE comes from robustness on the paths in which β is poorly identified and from shrinkage toward the prior; on a typical path, the MLE is equally accurate and, with an efficient implementation, equally fast at batch size one.

On real data, the Hawkes model captures most of the clustering of price changes in crypto perpetual futures, but not all of it, and its parameters occupy a different region of the parameter space, with null cross-excitation. Networks trained on MLE labels of real windows reproduce those labels well only when they start from the synthetic weights; fine-tuned, the LSTM and the Transformer are equivalent on the test block, and the Transformer degrades less under a change of regime. A pre-registered addendum on a new week showed that the selected LSTM was undertrained and that the average of the two networks is the most accurate estimator, and it measured how much of the decay the 100 ms grid hides; a network trained on the true parameters undoes that bias in simulation, but not yet verifiably on the market. On a second new week, an LSTM pretrained on a prior built from the real labels cut the loss by almost two thirds, the largest out-of-sample gain of this work. The Hawkes volatility implied by the MLE ranks the windows like the realized volatility, while the selected network's estimates keep its level but lose most of the ranking, part of which the Transformer recovers.

The answer to the research question is therefore negative for accuracy on synthetic data and open for robustness on real data. The automatic generation of tables and figures, the decision records and the single opening of each test block reduce the risk of divergence between artifacts and text and make the negative results as reliable as the positive ones.

A Ambiguities in Lee (2023)

The text mentions paths of 2,000 steps, but an input shape of 1,000. It does not publish the exact parameter ranges, seeds, batch size or NBBO data. This mention of 2,000 steps is kept only as historical context for the reconstruction. The final protocol fixes 1,000 events and does not compare lengths; reconstructed decisions are marked R2; crypto, A1; Transformer, E1. The paper calls the two LSTM layers followed by the dense layer “three layers”; the implementation follows this structure. The $\log(1 + \Delta t)$ input and the ± 1 encoding of direction are also R2.

B Computational Reproduction

```
uv sync --python 3.13 --extra dev
make test
make synthetic-final      # synthetic data with seed and SHA-256
make local-final          # resumable training, MLE, consolidation, figures and
    PDFs
make fixed-point          # E2 experiment at Lee's fixed point
make posthoc              # empirical calibration and complementary analysis
```

C Pipeline Validation History

Before the frozen protocol, reduced tests validated the integration of the components. Their numbers are not comparable to the final results and are recorded only for traceability.

Table C.1 – Integration tests prior to the frozen protocol.

Test	Configuration	Result
Reduced synthetic	1,024/128 paths, 128 events, 3 epochs	LSTM MSE 0.0448; Transformer 0.0328
Empirical integration	12 MLE-labeled windows, training only	Training MSE 0.5038 and 0.1800
Reduced frozen protocol	48/16/16 paths, 2 epochs	pipeline validated

References

AÏT-SAHALIA, Y.; CACHO-DIAZ, J.; LAEVEN, R. J. A. Modeling financial contagion using mutually exciting jump processes. *Journal of Financial Economics*, v. 117, n. 3, p. 585–606, 2015.

AKAIKE, H. A new look at the statistical model identification. *IEEE Transactions on Automatic Control*, v. 19, n. 6, p. 716–723, 1974.

ANDERSEN, T. G. et al. Modeling and forecasting realized volatility. *Econometrica*, v. 71, n. 2, p. 579–625, 2003.

BACRY, E. et al. Modelling microstructure noise with mutually exciting point processes. *Quantitative Finance*, v. 13, n. 1, p. 65–77, 2013.

BACRY, E.; MASTROMATTEO, I.; MUZY, J.-F. Hawkes processes in finance. *Market Microstructure and Liquidity*, v. 1, n. 1, p. 1550005, 2015.

BARNDORFF-NIELSEN, O. E.; SHEPHARD, N. Econometric analysis of realized volatility and its use in estimating stochastic volatility models. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, v. 64, n. 2, p. 253–280, 2002.

Binance. *Binance Public Data*. 2026. <https://data.binance.vision> e <https://github.com/binance/binance-public-data>. Acesso em 31 ago. 2026.

BOUCHAUD, J.-P. et al. *Trades, Quotes and Prices: Financial Markets Under the Microscope*. Cambridge: Cambridge University Press, 2018.

BOUTHILLIER, X. et al. Accounting for variance in machine learning benchmarks. In: SMOLA, A.; DIMAKIS, A.; STOICA, I. (Ed.). *Proceedings of Machine Learning and Systems*. [s.n.], 2021. v. 3, p. 747–769. Disponível em: <https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html>.

BOWSHER, C. G. Modelling security market events in continuous time: intensity based, multivariate point process models. *Journal of Econometrics*, v. 141, n. 2, p. 876–912, 2007.

BROWN, E. N. et al. The time-rescaling theorem and its application to neural spike train data analysis. *Neural Computation*, v. 14, n. 2, p. 325–346, 2002.

CHO, K. et al. On the properties of neural machine translation: encoder–decoder approaches. In: WU, D. et al. (Ed.). *Proceedings of SSST-8, Eighth Workshop on Syntax, Semantics and Structure in Statistical Translation*. Doha, Qatar: Association for Computational Linguistics, 2014. p. 103–111. ArXiv:1409.1259.

CRANMER, K.; BREHMER, J.; LOUPPE, G. The frontier of simulation-based inference. *Proceedings of the National Academy of Sciences*, v. 117, n. 48, p. 30055–30062, 2020.

DALEY, D. J.; VERE-JONES, D. *An Introduction to the Theory of Point Processes. Volume I: Elementary Theory and Methods*. Second. New York: Springer, 2003. (Probability and Its Applications).

DIEBOLD, F. X.; MARIANO, R. S. Comparing predictive accuracy. *Journal of Business & Economic Statistics*, v. 13, n. 3, p. 253–263, 1995.

DU, N. et al. Recurrent marked temporal point processes: embedding event history to vector. In: *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*. [S.l.]: ACM, 2016. p. 1555–1564.

EFRON, B.; TIBSHIRANI, R. J. *An Introduction to the Bootstrap*. New York: Chapman & Hall, 1993. (Monographs on Statistics and Applied Probability, 57).

EMBRECHTS, P.; LINIGER, T.; LIN, L. Multivariate Hawkes processes: an application to financial data. *Journal of Applied Probability*, v. 48A, p. 367–378, 2011.

FILIMONOV, V.; SORNETTE, D. Quantifying reflexivity in financial markets: toward a prediction of flash crashes. *Physical Review E*, v. 85, n. 5, p. 056108, 2012.

GERBER, F.; NYCHKA, D. Fast covariance parameter estimation of spatial Gaussian process models using neural networks. *Stat*, v. 10, n. 1, p. e382, 2021.

GOURIÉROUX, C.; MONFORT, A.; RENAULT, E. Indirect inference. *Journal of Applied Econometrics*, v. 8, n. S1, p. S85–S118, 1993.

HARDIMAN, S. J.; BERCOT, N.; BOUCHAUD, J.-P. Critical reflexivity in financial markets: a Hawkes process analysis. *The European Physical Journal B*, v. 86, n. 10, p. 442, 2013.

HAWKES, A. G. Point spectra of some mutually exciting point processes. *Journal of the Royal Statistical Society: Series B (Methodological)*, v. 33, n. 3, p. 438–443, 1971.

HAWKES, A. G. Spectra of some self-exciting and mutually exciting point processes. *Biometrika*, v. 58, n. 1, p. 83–90, 1971.

HAWKES, A. G. Hawkes processes and their applications to finance: a review. *Quantitative Finance*, v. 18, n. 2, p. 193–198, 2018.

HE, S. et al. *Fundamentals of perpetual futures*. 2022. ArXiv preprint arXiv:2212.06888. Working paper; latest revision v7, 17 Sep. 2026.

HOCHREITER, S.; SCHMIDHUBER, J. Long short-term memory. *Neural Computation*, v. 9, n. 8, p. 1735–1780, 1997.

HOLM, S. A simple sequentially rejective multiple test procedure. *Scandinavian Journal of Statistics*, v. 6, n. 2, p. 65–70, 1979.

KINGMA, D. P.; BA, J. Adam: A method for stochastic optimization. In: *3rd International Conference on Learning Representations (ICLR 2015)*. San Diego: [s.n.], 2015. ArXiv:1412.6980. Disponível em: <https://arxiv.org/abs/1412.6980>.

KROGH, A.; VEDELSBY, J. Neural network ensembles, cross validation, and active learning. In: *Advances in Neural Information Processing Systems 7*. Cambridge, MA: MIT Press, 1995. p. 231–238.

LAMBE, J. J. et al. Neural networks for parameter estimation of the discretely observed Hawkes process. *Journal of Computational and Graphical Statistics*, 2026. Published online 27 October 2026; volume and pages not yet assigned. Preprint: arXiv:2506.01258.

LAUB, P. J.; LEE, Y.; TAIMRE, T. *The Elements of Hawkes Processes*. Cham: Springer, 2021.

LEE, K. Recurrent neural network based parameter estimation of Hawkes model on high-frequency financial data. *Finance Research Letters*, v. 55, p. 103922, 2023.

LEE, K. Application of Hawkes volatility in the observation of filtered high-frequency price process in tick structures. *Applied Stochastic Models in Business and Industry*, v. 40, n. 6, p. 1689–1711, 2024.

LEE, K.; SEO, B. K. Marked Hawkes process modeling of price dynamics and volatility estimation. *Journal of Empirical Finance*, v. 40, p. 174–200, 2017.

LEHMANN, E. L.; CASELLA, G. *Theory of Point Estimation*. Second. New York: Springer, 1998. (Springer Texts in Statistics).

LENZI, A. et al. Neural networks for parameter estimation in intractable models. *Computational Statistics & Data Analysis*, v. 185, p. 107762, 2023.

LOSHCHILOV, I.; HUTTER, F. SGDR: Stochastic gradient descent with warm restarts. In: *5th International Conference on Learning Representations (ICLR 2017)*. [s.n.], 2017. ArXiv:1608.03983. Disponível em: <https://arxiv.org/abs/1608.03983>.

MAKAROV, I.; SCHOAR, A. Trading and arbitrage in cryptocurrency markets. *Journal of Financial Economics*, v. 135, n. 2, p. 293–319, 2020.

MEI, H.; EISNER, J. M. The neural Hawkes process: a neurally self-modulating multivariate point process. In: GUYON, I. et al. (Ed.). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2017. v. 30. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2017/hash/6463c88460bd63bbe256e495c63aa40b-Abstract.html>.

OGATA, Y. The asymptotic behaviour of maximum likelihood estimators for stationary point processes. *Annals of the Institute of Statistical Mathematics*, v. 30, n. 2, p. 243–261, 1978.

OGATA, Y. On Lewis' simulation method for point processes. *IEEE Transactions on Information Theory*, v. 27, n. 1, p. 23–31, 1981.

OGATA, Y. Statistical models for earthquake occurrences and residual analysis for point processes. *Journal of the American Statistical Association*, v. 83, n. 401, p. 9–27, 1988.

OZAKI, T. Maximum likelihood estimation of Hawkes' self-exciting point processes. *Annals of the Institute of Statistical Mathematics*, v. 31, n. 1, p. 145–155, 1979.

PRIMO, G. O. *Padrão de aglomeração intradiário do mercado de ações brasileiro: análise exploratória com processo de Hawkes*. Dissertação (Dissertação (Mestrado em Economia)) — Programa de Pós-Graduação em Economia, Universidade Federal de Pernambuco, Recife, 2018. Disponível em: <https://repositorio.ufpe.br/handle/123456789/32068>.

RADEV, S. T. et al. BayesFlow: learning complex stochastic models with invertible neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, v. 33, n. 4, p. 1452–1466, 2022.

SAINSBURY-DALE, M.; ZAMMIT-MANGION, A.; HUSER, R. Likelihood-free parameter estimation with neural Bayes estimators. *The American Statistician*, v. 78, n. 1, p. 1–14, 2024.

SELF, S. G.; LIANG, K.-Y. Asymptotic properties of maximum likelihood estimators and likelihood ratio tests under nonstandard conditions. *Journal of the American Statistical Association*, v. 82, n. 398, p. 605–610, 1987.

SHCHUR, O. et al. Neural temporal point processes: a review. In: *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*. [S.l.]: International Joint Conferences on Artificial Intelligence Organization, 2021. p. 4585–4593.

SILVA, L. A. d. O. *Processos de Hawkes e aplicações*. Dissertação (Dissertação (Mestrado em Matemática Aplicada)) — Instituto de Matemática, Estatística e Ciência da Computação, Universidade de São Paulo, São Paulo, 2026.

TADDEO, M. M.; SALVI, L.; RIVERA-CASTRO, M. A. Sobre o uso de processos de Hawkes para avaliar a frequência de saltos nos log retornos de criptomoedas. *Brazilian Review of Finance*, v. 22, n. 2, p. 69–92, 2024.

TOKE, I. M. *An Introduction to Hawkes Processes with Applications to Finance*. 2011. Lecture slides, École Centrale Paris, BNP Paribas Chair of Quantitative Finance. Slides dated 4 February 2011.

VASWANI, A. et al. Attention is all you need. In: GUYON, I. et al. (Ed.). *Advances in Neural Information Processing Systems*. Curran Associates, Inc., 2017. v. 30. Disponível em: <https://proceedings.neurips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>.

WILCOXON, F. Individual comparisons by ranking methods. *Biometrics Bulletin*, v. 1, n. 6, p. 80–83, 1945.

XIONG, R. et al. On layer normalization in the Transformer architecture. In: DAUMÉ III, H.; SINGH, A. (Ed.). *Proceedings of the 37th International Conference on Machine Learning*. PMLR, 2020. (Proceedings of Machine Learning Research, v. 119), p. 10524–10533. Disponível em: <https://proceedings.mlr.press/v119/xiong20b.html>.

ZHANG, L.; MYKLAND, P. A.; AÏT-SAHALIA, Y. A tale of two time scales: determining integrated volatility with noisy high-frequency data. *Journal of the American Statistical Association*, v. 100, n. 472, p. 1394–1411, 2005.

ZHANG, Q. et al. Self-attentive Hawkes process. In: DAUMÉ III, H.; SINGH, A. (Ed.). *Proceedings of the 37th International Conference on Machine Learning*. PMLR, 2020. (Proceedings of Machine Learning Research, v. 119), p. 11183–11193. Disponível em: <https://proceedings.mlr.press/v119/zhang20q.html>.

ZUO, S. et al. Transformer Hawkes process. In: DAUMÉ III, H.; SINGH, A. (Ed.). *Proceedings of the 37th International Conference on Machine Learning*. PMLR, 2020. (Proceedings of Machine Learning Research, v. 119), p. 11692–11702. Disponível em: <https://proceedings.mlr.press/v119/zuo20a.html>.